

```
/* ===== STATE ===== */
const STORAGE_KEY = 'splitCoachData_v1';

function defaultState(){
  return {
    settings: {
      name: 'Matty',
      weightUnit: 'stone', // 'stone' or
      'kg'
      weightStone: 11, weightLb: 11,
weightKg: null,
      heightCm: 165, // 5'5"
      age: 46,
      sex: 'male',
      raceName: 'New York City Marathon',
      raceDate: '2026-11-01',
      longestRunMiles: 16,
      easyPaceMinPerMile: 10.5,
      distanceUnit: 'mi', // 'mi' or 'km'
      maxHR: null, restingHR: null,
      voiceReplies: true,
      dayShiftHours: '7am-5pm',
eveningShiftHours: '1pm-11pm',
nightShiftHours: '11pm-7am',
      crossTrainDefault: 'Stationary bike',
      apiKey: ''
    },
    shifts: {}, // { 'YYYY-MM-DD':
'off'|'day'|'evening'|'night' }
    checkins: {}, // { 'YYYY-MM-DD':
```

```

    {mood, energy, soreness, note} }
    log: {}, // { 'YYYY-MM-DD':
{done:true, distance, notes} }
    overrides: {}, // { 'YYYY-MM-DD':
{type, desc, badge} } manual plan edits
    chat: [], // [{role, content,
ts}]
    mealIdeas: {}, // { 'YYYY-MM-DD':
text } AI meal suggestions cache
    foodLog: {}, // { 'YYYY-MM-DD':
[{barcode, name, grams, kcal, protein,
carbs, fat, loggedAt}] }
    races: [{
        id: 'r1', name: 'New York City
Marathon', date: '2026-11-01',
        goalHours: 4, goalMinutes: 30,
        recentDistance: 13.2,
recentTimeMinutes: 125,
        done: false
    }],
    activeRaceId: 'r1'
};
}

let state = loadState();

function loadState(){
    try{
        const raw =
localStorage.getItem(STORAGE_KEY);
        if(!raw) return defaultState();
        const parsed = JSON.parse(raw);
        // merge with defaults to survive
        schema growth
    }
}

```

```

    const d = defaultState();
    return {
      settings: Object.assign({},
d.settings, parsed.settings || {}),
      shifts: parsed.shifts || {},
      checkins: parsed.checkins || {},
      log: parsed.log || {},
      overrides: parsed.overrides || {},
      chat: parsed.chat || [],
      races: (parsed.races &&
parsed.races.length) ? parsed.races :
d.races,
      activeRaceId: parsed.activeRaceId ||
d.activeRaceId,
      mealIdeas: parsed.mealIdeas || {},
      foodLog: parsed.foodLog || {}
    };
  }catch(e){
    console.error('Failed to load state,
starting fresh', e);
    return defaultState();
  }
}

function saveState(){
  localStorage.setItem(STORAGE_KEY,
JSON.stringify(state));
}

/* ===== DATE HELPERS =====
*/
function fmtDate(d){
  const y=d.getFullYear(),
m=String(d.getMonth()+1).padStart(2,'0'),

```

```

day=String(d.getDate()).padStart(2,'0');
return `${y}-${m}-${day}`;
}
function todayKey(){ return fmtDate(new
Date()); }
function parseKey(k){ const
[y,m,d]=k.split('-').map(Number); return
new Date(y, m-1, d); }
function addDays(d, n){ const nd = new
Date(d); nd.setDate(nd.getDate()+n); return
nd; }
function startOfWeek(d){ // Monday start
const nd = new Date(d);
const day = nd.getDay(); // 0=Sun
const diff = (day === 0 ? -6 : 1 - day);
nd.setDate(nd.getDate()+diff);
nd.setHours(0,0,0,0);
return nd;
}
function daysBetween(a,b){ return
Math.round((b-a)/(1000*60*60*24)); }

/* ===== RACE HELPERS =====
*/
function getActiveRace(){
  if(!state.races ||
state.races.length===0){
    state.races = defaultState().races;
    state.activeRaceId = state.races[0].id;
    saveState();
  }
  return
state.races.find(r=>r.id===state.activeRace
Id) || state.races[0];

```

```

}
function activeRaceDate(){ return
parseKey(getActiveRace().date); }
function addRace(name, date){
  const id = 'r' + Date.now();
  state.races.push({ id, name, date,
goalHours:null, goalMinutes:null,
recentDistance:null,
recentTimeMinutes:null, done:false });
  state.activeRaceId = id;
  saveState();
  return id;
}
function setActiveRace(id){
state.activeRaceId = id; saveState();
render(); }
function markRaceDone(id){
  const r = state.races.find(r=>r.id===id);
  if(r) r.done = true;
  saveState(); render();
}
function deleteRace(id){
  state.races =
state.races.filter(r=>r.id!==id);
  if(state.races.length===0) state.races =
defaultState().races;
  if(state.activeRaceId===id)
state.activeRaceId = state.races[0].id;
  saveState(); render();
}
function goalTotalMinutes(race){
  race = race || getActiveRace();
  if(race.goalHours===null &&
race.goalMinutes===null) return null;

```

```

    return (Number(race.goalHours)||0)*60 +
    (Number(race.goalMinutes)||0);
  }
  function predictedMarathonMinutes(race){
    race = race || getActiveRace();
    if(race.recentDistance &&
    race.recentTimeMinutes){
      return race.recentTimeMinutes *
    Math.pow(26.2/Number(race.recentDistance),
    1.06);
    }
    return null;
  }
  function goalPaceMinPerMile(race){
    const total = goalTotalMinutes(race);
    return total ? total/26.2 : null;
  }
  function predictedPaceMinPerMile(race){
    const pred =
    predictedMarathonMinutes(race);
    return pred ? pred/26.2 : null;
  }
  function fmtMinutesAsHM(mins){
    if(mins==null) return '-';
    const h = Math.floor(mins/60), m =
    Math.round(mins%60);
    return `${h}h
    ${String(m).padStart(2,'0')}m`;
  }
  function raceRealism(race){
    race = race || getActiveRace();
    const goal = goalTotalMinutes(race);
    const pred =
    predictedMarathonMinutes(race);

```

```

    if(!goal) return { verdict:'no-goal',
text:'Set a goal time to see how realistic
it is.' };
    if(!pred) return { verdict:'unknown',
text:'Add a recent race or time-trial
result (e.g. a 10K or half marathon time)
so the coach can judge how realistic this
goal is.' };
    const diffPct = ((goal - pred) / pred) *
100;
    const wOut = weeksToRace(race);
    let verdict, text;
    if(diffPct >= 4){
        verdict = 'conservative';
        text = `Comfortably realistic – your
recent form predicts around
${fmtMinutesAsHM(pred)}, faster than your
${fmtMinutesAsHM(goal)} goal. You've got
room to hold back early and negative-
split.`;
    } else if(diffPct >= -3){
        verdict = 'realistic';
        text = `Realistic – your recent form
predicts almost exactly your
${fmtMinutesAsHM(goal)} goal (predicted
${fmtMinutesAsHM(pred)}). A solid build and
smart pacing should get you there.`;
    } else if(diffPct >= -9){
        verdict = 'ambitious';
        text = `Ambitious but possible – your
recent form predicts
${fmtMinutesAsHM(pred)}, about
${Math.abs(Math.round(diffPct))}% slower
than your ${fmtMinutesAsHM(goal)} goal.

```

```

With ${wOut} weeks to go, a strong peak and
disciplined taper could close that gap, but
have a backup pacing plan.`;
    } else {
        verdict = 'unrealistic';
        text = `A stretch right now – your
recent form predicts
${fmtMinutesAsHM(pred)}, well outside your
${fmtMinutesAsHM(goal)} goal
(${Math.abs(Math.round(diffPct))}% off)
with ${wOut} weeks to go. Worth either
adjusting the goal toward
${fmtMinutesAsHM(pred)}, or treating this
goal as a longer-term target for a future
race.`;
    }
    return { verdict, text, goal, pred,
diffPct };
}
function trainingPaces(race){
    race = race || getActiveRace();
    const base = goalPaceMinPerMile(race) ||
predictedPaceMinPerMile(race);
    if(!base) return null;
    return {
        marathon: base,
        easy: base + 1.25,
        tempo: base - 0.35,
        interval: base - 0.75
    };
}
function weeksToRace(race){
    const today = new Date();
    today.setHours(0,0,0,0);

```



```

    const raceDate =
    parseKey((race||getActiveRace()).date);
    const diff = daysBetween(today,
    raceDate);
    return Math.ceil(diff/7);
  }
  function dowShort(d){ return
  ['Sun','Mon','Tue','Wed','Thu','Fri','Sat']
  [d.getDay()]; }
  function monthDayLabel(d){ return
  `${d.getDate()}
  $[['Jan','Feb','Mar','Apr','May','Jun','Jul
  ','Aug','Sep','Oct','Nov','Dec']]
  [d.getMonth()]}`; }

  /* ===== UNIT HELPERS =====
  */
  function getWeightKg(){
    const s = state.settings;
    if(s.weightUnit === 'kg') return
    s.weightKg || 70;
    const totalLb = (s.weightStone||0)*14 +
    (s.weightLb||0);
    return totalLb * 0.453592;
  }
  function getWeightLb(){ return
  getWeightKg() * 2.20462; }

  /* ===== UNIT DISPLAY HELPERS
  ===== */
  const MI_TO_KM = 1.60934;
  function isKm(){ return
  state.settings.distanceUnit === 'km'; }
  function formatDistance(miles){

```

```

    if(!miles) return '0' +
(isKm()?'km':'mi');
    if(isKm()){
        const km = miles * MI_TO_KM;
        return `${km>=10 ? Math.round(km) :
Math.round(km*10)/10}km`;
    }
    return `${miles}mi`;
}
function fmtPace(minPerMile){
    if(minPerMile==null) return '-';
    let val = minPerMile, unit = 'mi';
    if(isKm()){ val = minPerMile / MI_TO_KM;
unit = 'km'; }
    const m = Math.floor(val), s =
Math.round((val-m)*60);
    return
`${m}:${String(s).padStart(2,'0')}/${unit}`
;
}
// Converts wording like "16 miles" or "1
mile" found inside generated session text
// into the athlete's preferred unit for
display, without altering stored data.
function convertDistanceWording(desc){
    if(!desc || !isKm()) return desc;
    return desc.replace(/(\d+(?:\.\d+)?)\s*
(miles|mile|mi)\b/gi, (match, num)=>{
        const km = parseFloat(num) * MI_TO_KM;
        const rounded = km >= 10 ?
Math.round(km) : Math.round(km*10)/10;
        return `${rounded}km`;
    }));
}

```

```

function paceMileToDisplayValue(minPerMile)
{
    return isKm() ? (minPerMile / MI_TO_KM) :
minPerMile;
}
function paceDisplayValueToMile(displayVal)
{
    return isKm() ? (displayVal * MI_TO_KM) :
displayVal;
}

/* ===== HEART RATE ZONES (Karvonen)
===== */
function hrZones(){
    const s = state.settings;
    if(!s.maxHR || !s.restingHR) return null;
    const reserve = s.maxHR - s.restingHR;
    const zone = (loPct, hiPct) => ({
        lo: Math.round(s.restingHR +
reserve*loPct),
        hi: Math.round(s.restingHR +
reserve*hiPct)
    });
    return {
        z1: zone(0.5, 0.6), z2: zone(0.6, 0.7),
z3: zone(0.7, 0.8),
        z4: zone(0.8, 0.9), z5: zone(0.9, 1.0)
    };
}

/* ===== SHIFT HELPERS =====
*/
const SHIFT_COLORS = { off: '#22c55e',
day: '#3aa0ff', evening: '#ff4d4d',

```

```

night: '#a855f7', unknown: '#5a6478' };
const SESSION_DOT_COLORS = {
  long: '#ff4d4d', quality: '#3aa0ff',
  easy: '#22c55e', cross: '#3aa0ff',
  rest: '#5a6478', race: '#ff4d4d' };
function dayDots(dateKey){
  const shift = getShift(dateKey);
  const eff = getEffectiveSession(dateKey);
  const dots = [ `
```

```

evening: s.eveningShiftHours, night:
s.nightShiftHours }[shiftType];
  return hours ?
`${SHIFT_LABELS[shiftType]} (${hours})` :
SHIFT_LABELS[shiftType];
}
const SHIFT_LABELS = { off:'Off', day:'Day
shift', evening:'Evening shift',
night:'Night shift', unknown:'Not set' };
function getShift(dateKey){ return
state.shifts[dateKey] || 'unknown'; }

/* ===== TRAINING PLAN ENGINE
===== */
// Template keyed by weeks-out (7 = far
out/build, 0 = race week)
const STRENGTH_BLOCKS = {
  build: {
    label: 'Runner strength – build phase
(30-40min)',
    exercises: ['Squats or goblet squats:
3x8', 'Romanian deadlifts (single-leg or
double): 3x8 each side', 'Walking lunges:
3x10 each leg', 'Glute bridges: 3x12',
'Plank: 3x40sec', 'Side plank: 2x30sec each
side', 'Calf raises: 3x15']
  },
  peak: {
    label: 'Runner strength – peak phase,
maintain not build (25-30min)',
    exercises: ['Squats: 2x8 (moderate
load, not to failure)', 'Single-leg RDL:
2x8 each side', 'Glute bridges: 2x12',
'Plank: 2x40sec', 'Calf raises: 2x15', 'Hip

```

```

clamshells or band walks: 2x12 each side']
  },
  taper: {
    label: 'Runner strength – light
activation only (15-20min)',
    exercises: ['Bodyweight squats: 2x10',
'Glute bridges: 2x12', 'Clamshells: 2x10
each side', 'Plank: 2x30sec', 'Ankle
mobility + calf raises: 2x12']
  },
  race: {
    label: 'Light mobility only – no loaded
strength work',
    exercises: ['Dynamic leg swings: 1min
each leg', 'Hip circles: 1min', 'Gentle
calf and hamstring stretches']
  }
};
const WEEK_TEMPLATES = {
  8: { phase:'Build', long:16, quality:
{desc:'Tempo run: 2mi easy + 4mi @ steady-
hard effort + 1mi easy', badge:'quality',
segments:[
  {label:'Warm-up', kind:'distance',
miles:2, paceType:'easy'},
  {label:'Tempo segment',
kind:'distance', miles:4,
paceType:'tempo'},
  {label:'Cool-down', kind:'distance',
miles:1, paceType:'easy'}
]}, easyCount:2, cross:1,
strengthCount:2, strengthBlock:'build' },
  7: { phase:'Build', long:16, quality:
{desc:'Tempo run: 2mi easy + 4mi @ steady-

```

```

hard effort + 1mi easy', badge:'quality',
segments:[
  {label:'Warm-up', kind:'distance',
miles:2, paceType:'easy'},
  {label:'Tempo segment',
kind:'distance', miles:4,
paceType:'tempo'},
  {label:'Cool-down', kind:'distance',
miles:1, paceType:'easy'}
]], easyCount:2, cross:1,
strengthCount:2, strengthBlock:'build' },
6: { phase:'Build', long:18, quality:
{desc:'5 x 1 mile @ 10K effort, 3min jog
recovery between', badge:'quality',
segments:[
  {label:'Warm-up', kind:'distance',
miles:1.5, paceType:'easy'},
  {label:'Main set', kind:'reps',
reps:5, repDistance:1, paceType:'interval',
recovery:'3 min easy jog recovery between
reps'},
  {label:'Cool-down', kind:'distance',
miles:1, paceType:'easy'}
]], easyCount:2, cross:1,
strengthCount:2, strengthBlock:'build' },
5: { phase:'Step-back', long:13, quality:
{desc:'Tempo run: 1mi easy + 5mi @ steady-
hard effort + 1mi easy', badge:'quality',
segments:[
  {label:'Warm-up', kind:'distance',
miles:1, paceType:'easy'},
  {label:'Tempo segment',
kind:'distance', miles:5,
paceType:'tempo'},

```

```

        {label:'Cool-down', kind:'distance',
miles:1, paceType:'easy'}
    ]}, easyCount:2, cross:1,
strengthCount:2, strengthBlock:'build' },
    4: { phase:'Peak', long:20, quality:
{desc:'3 x 2 miles @ half-marathon effort,
4min jog recovery', badge:'quality',
segments:[
        {label:'Warm-up', kind:'distance',
miles:1.5, paceType:'easy'},
        {label:'Main set', kind:'reps',
reps:3, repDistance:2, paceType:'tempo',
recovery:'4 min easy jog recovery between
reps'},
        {label:'Cool-down', kind:'distance',
miles:1, paceType:'easy'}
    ]}, easyCount:2, cross:1,
strengthCount:1, strengthBlock:'peak' },
    3: { phase:'Taper begins', long:13,
quality:{desc:'4 miles easy with 4 x 20sec
strides at the end', badge:'easy',
segments:[
        {label:'Main run', kind:'distance',
miles:4, paceType:'easy'},
        {label:'Strides', kind:'custom',
customText:'4 x 20 seconds at a quick,
relaxed sprint (not all-out) with full
recovery between – sharpens turnover
without adding fatigue.'}
    ]}, easyCount:2, cross:1,
strengthCount:1, strengthBlock:'peak' },
    2: { phase:'Taper', long:10, quality:
{desc:'4 miles @ marathon race pace, rest
easy', badge:'quality', segments:[

```



```

        {label:'Warm-up', kind:'distance',
miles:1, paceType:'easy'},
        {label:'Marathon-pace segment',
kind:'distance', miles:4,
paceType:'marathon'},
        {label:'Cool-down', kind:'distance',
miles:1, paceType:'easy'}
    ]}, easyCount:2, cross:1,
strengthCount:1, strengthBlock:'taper' },
    1: { phase:'Final taper', long:6,
quality:{desc:'2 miles easy + 4 x 20sec
strides', badge:'easy', segments:[
        {label:'Main run', kind:'distance',
miles:2, paceType:'easy'},
        {label:'Strides', kind:'custom',
customText:'4 x 20 seconds at a quick,
relaxed sprint with full recovery
between.'}
    ]}, easyCount:2, cross:0,
strengthCount:1, strengthBlock:'taper' },
    0: { phase:'Race week', long:3, quality:
{desc:'Rest – save it for race day',
badge:'rest', segments:[
        {label:'Today', kind:'custom',
customText:'Complete rest – save your legs
for race day.'}
    ]}, easyCount:1, cross:0,
strengthCount:0, strengthBlock:'race' }
};
function templateForWeeksOut(n){
    if(n >= 8) return WEEK_TEMPLATES[8];
    if(n <= 0) return WEEK_TEMPLATES[0];
    return WEEK_TEMPLATES[n];
}

```

```

// Build a 7-day schedule for the week
containing `anchorDate`, using shift data.
function buildWeekSchedule(weekStart){
  const race = getActiveRace();
  const raceDate = parseKey(race.date);
  const dates = [];
  for(let i=0;i<7;i++)
    dates.push(addDays(weekStart, i));

  // weeks-out relative to THIS week (not
  just today)
  const thisWeekOutRaw =
    Math.ceil(daysBetween(weekStart,
    raceDate)/7);
  const tmp1 =
    templateForWeeksOut(thisWeekOutRaw);

  const shiftStatuses = dates.map(d =>
    getShift(fmtDate(d)));
  const isRaceWeek = thisWeekOutRaw <= 0;

  // scoring for placing key sessions:
  prefer 'off', then 'day', then 'evening',
  avoid 'night'/'unknown'
  // 'off' is best. 'evening' shift days
  leave the morning free for a proper run, so
  they're
  // nearly as good as a day off. 'day'
  shift usually means an early start, so
  mornings are
  // tighter. 'night' shift needs
  recovery/sleep, so it's actively avoided
  for hard sessions.

```

```

    const score = s => ({off:3, evening:2.5,
day:1, unknown:0.5, night:-2}[s] ?? 0);

    const assigned = new Array(7).fill(null);

    // Race day itself if race week and
today's week contains race date
    let raceIdx = -1;
    if(isRaceWeek){
        dates.forEach((d,i)=>{ if(fmtDate(d)
=== fmtDate(raceDate)) raceIdx = i; });
    }
    if(raceIdx >= 0){
        assigned[raceIdx] = { type:'race',
badge:'long', desc:`RACE DAY –
${race.name}`, miles: 26.2 };
    }

    // pick long run day: best scoring day,
prefer Sat/Sun, excluding race day
    let candidates =
dates.map((d,i)=>i).filter(i=>i!==raceIdx);
    candidates.sort((a,b)=>{
        const sd = score(shiftStatuses[b]) -
score(shiftStatuses[a]);
        if(sd !== 0) return sd;
        // prefer weekend
        const wa =
[6,0].includes(dates[a].getDay()) ? 1:0;
        const wb =
[6,0].includes(dates[b].getDay()) ? 1:0;
        return wb - wa;
    });
    const longIdx = candidates[0];

```

```

    if(longIdx !== undefined &&
assigned[longIdx] === null){
        assigned[longIdx] = { type:'long',
badge:'long', desc:`Long run – ${tpl.long}
miles, easy conversational pace`, miles:
tpl.long };
    }

    // pick quality day: next best scoring
day, not adjacent to long run if possible,
not night shift
    let qCandidates =
candidates.filter(i=>i!==longIdx &&
shiftStatuses[i] !== 'night');
    qCandidates.sort((a,b)=>{
        const adjA = Math.abs(a-longIdx)<=1 ?
1:0;
        const adjB = Math.abs(b-longIdx)<=1 ?
1:0;
        if(adjA !== adjB) return adjA-adjB;
        return score(shiftStatuses[b]) -
score(shiftStatuses[a]);
    });
    const qIdx = qCandidates[0];
    if(qIdx !== undefined && assigned[qIdx]
=== null){
        assigned[qIdx] = { type:'quality',
badge: tpl.quality.badge, desc:
tpl.quality.desc, miles: null, segments:
tpl.quality.segments };
    }

    // remaining days: fill easy runs
(easyCount), cross-train (cross), night-

```

```

shift -> rest, rest fill rest
  let remaining =
dates.map((d,i)=>i).filter(i=>assigned[i]==
=null);
  let easyLeft = tpl.easyCount;
  let crossLeft = tpl.cross;

  // Night shift days become rest by
default (protect sleep/recovery)
  remaining = remaining.filter(i=>{
    if(shiftStatuses[i] === 'night'){
      assigned[i] = { type:'rest',
badge:'rest', desc:'Rest – night shift.
Prioritise sleep.', miles:0 };
      return false;
    }
    return true;
  });

  remaining.sort((a,b)=>
score(shiftStatuses[b]) -
score(shiftStatuses[a]));
  remaining.forEach(i=>{
    if(easyLeft > 0 && assigned[i]===null){
      assigned[i] = { type:'easy',
badge:'easy', desc:'Easy run – 3-4 miles,
comfortable pace', miles:3.5 };
      easyLeft--;
    } else if(crossLeft > 0 &&
assigned[i]===null){
      assigned[i] = { type:'cross',
badge:'cross', desc:`Cross-train –
${state.settings.crossTrainDefault}, 40min
moderate effort`, miles:0 };

```

```

        crossLeft--;
    } else if(assigned[i]===null){
        assigned[i] = { type:'rest',
badge:'rest', desc:'Rest day', miles:0 };
    }
    });

    // Strength: attach to days that are
    easy/cross/rest (not long, quality, race,
    night-shift-rest-days already fine too)
    let strengthLeft = tpl.strengthCount ||
0;
    if(strengthLeft > 0){
        const block =
STRENGTH_BLOCKS[tpl.strengthBlock] ||
STRENGTH_BLOCKS.build;
        const strengthCandidates =
dates.map((d,i)=>i).filter(i=>{
            const t = assigned[i] &&
assigned[i].type;
            return (t === 'easy' || t === 'cross'
|| t === 'rest') && shiftStatuses[i] !==
'night' && i !== longIdx;
        });
        // spread them out rather than
clustering
        strengthCandidates.forEach((i, idx)=>{
            if(strengthLeft <= 0) return;
            if(idx % 2 === 0 ||
strengthCandidates.length <= strengthLeft){
                assigned[i].strength = { label:
block.label, exercises: block.exercises };
                strengthLeft--;
            }
        }
    }

```

```

    });
    // if still left (not enough spread-out
    slots), just fill remaining candidates in
    order
    for(const i of strengthCandidates){
        if(strengthLeft <= 0) break;
        if(!assigned[i].strength){
            assigned[i].strength = { label:
block.label, exercises: block.exercises };
            strengthLeft--;
        }
    }
}

return dates.map((d,i)=>({
    date: d,
    dateKey: fmtDate(d),
    shift: shiftStatuses[i],
    session: assigned[i],
    phase: tmp1.phase,
    weeksOut: thisWeekOutRaw
})));
}

function getSessionForDate(dateKey){
    const d = parseKey(dateKey);
    const ws = startOfWeek(d);
    const week = buildWeekSchedule(ws);
    return week.find(x=>x.dateKey ===
dateKey);
}

// Apply manual override or mood-based
adjustment

```

```

function getEffectiveSession(dateKey){
  const base = getSessionForDate(dateKey);
  if(!base) return null;
  if(state.overrides[dateKey]){
    return Object.assign({}, base, {
      session: state.overrides[dateKey],
      overridden:true });
  }
  const checkin = state.checkins[dateKey];
  if(checkin && base.session.type !==
    'rest' && base.session.type !== 'race'){
    const bad = (checkin.energy <= 2) ||
    (checkin.soreness >= 4);
    if(bad){
      let adj;
      if(base.session.type === 'long' ||
        base.session.type === 'quality'){
        adj = { type:'easy', badge:'easy',
          desc:`Adjusted: easy 25-30min instead of
            planned session – listen to your body
            today`, miles:3, adjusted:true, original:
            base.session.desc };
      } else if(base.session.type ===
        'easy'){
        adj = { type:'rest', badge:'rest',
          desc:'Adjusted: rest – recover today',
          miles:0, adjusted:true, original:
            base.session.desc };
      } else {
        adj = base.session;
      }
      return Object.assign({}, base, {
        session: adj });
    }
  }
}

```



```

    }
    return base;
}

/* ===== NUTRITION ENGINE
===== */
function estimateRunKcal(miles){
    if(!miles) return 0;
    return Math.round(miles * getWeightLb() *
0.63);
}
function estimateRunMinutes(miles){
    if(!miles) return 0;
    return Math.round(miles *
state.settings.easyPaceMinPerMile);
}
function bmr(){
    const s = state.settings;
    const w = getWeightKg(), h = s.heightCm,
a = s.age;
    const base = 10*w + 6.25*h - 5*a;
    return Math.round(s.sex === 'male' ? base
+ 5 : base - 161);
}
function nutritionForDay(dateKey){
    const eff = getEffectiveSession(dateKey);
    const type = eff ? eff.session.type :
'rest';
    const miles = sessionTotalMiles(eff &&
eff.session);
    const w = getWeightKg();

    const activityMultiplier = {rest:1.35,
cross:1.5, easy:1.5, quality:1.6, long:1.6,

```

```

    race:1.6}[type] ?? 1.4;
    const baseTDEE = bmr() *
activityMultiplier;
    const runKcal = estimateRunKcal(miles);
    const totalKcal = Math.round(baseTDEE +
runKcal);

    const carbsPerKg = {rest:3.5, cross:5,
easy:5, quality:7, long:8, race:8}[type] ??
4;
    const proteinPerKg = 1.8;
    const carbsG = Math.round(carbsPerKg *
w);
    const proteinG = Math.round(proteinPerKg
* w);
    const carbsKcal = carbsG*4, proteinKcal =
proteinG*4;
    const fatKcal = Math.max(totalKcal -
carbsKcal - proteinKcal, w*0.6*9);
    const fatG = Math.round(fatKcal/9);

    const minutes =
estimateRunMinutes(miles);
    const waterMl = Math.round(w*35 +
minutes*11); // ~660ml/hour during training
    const needsElectrolytes = minutes >= 75
|| type === 'long' || type === 'race';

    return { type, miles, totalKcal, carbsG,
proteinG, fatG, waterMl, needsElectrolytes,
minutes, runKcal, baseTDEE:
Math.round(baseTDEE) };
}

```

```

/* ===== POST-RUN RECOVERY
===== */
function recoveryForSession(dateKey){
  const eff = getEffectiveSession(dateKey);
  const miles = eff ?
sessionTotalMiles(eff.session) : 0;
  if(!miles) return null;
  const minutes =
estimateRunMinutes(miles);
  const w = getWeightKg();
  return {
    miles, minutes,
    carbsG: Math.round(w * 1.0),    //
~1g/kg quick-digesting carbs, standard
post-endurance guidance
    proteinG: Math.round(w * 0.3),  //
roughly 20-25g for most bodyweights
    fluidMl: Math.round(minutes * 11 *
1.5), // 150% of estimated sweat loss
during the run
    sodiumMg: minutes >= 60 ? 500 : 300
  };
}

/* ===== STRENGTH CHECKLIST
===== */
function toggleStrengthExercise(dateKey,
idx){
  if(!state.log[dateKey])
state.log[dateKey] = {};
  if(!state.log[dateKey].strengthChecks)
state.log[dateKey].strengthChecks = [];
  state.log[dateKey].strengthChecks[idx] =
!state.log[dateKey].strengthChecks[idx];
}

```

```

    saveState();
    render();
  }
  function renderStrengthChecklist(dateKey,
  strengthObj){
    const checks = (state.log[dateKey] &&
  state.log[dateKey].strengthChecks) || [];
    const doneCount =
  strengthObj.exercises.filter((_,i)=>checks[
  i]).length;
    return `
      <div style="font-size:12px; color:var(-
  -lane-dim); font-family:'Helvetica
  Neue',Arial,sans-serif; margin-
  bottom:10px;">${doneCount}/${strengthObj.ex
  ercises.length} done</div>
      <div>
        ${strengthObj.exercises.map((ex,i)=>`
          <div style="display:flex; align-
  items:center; gap:12px; padding:9px 0;
  border-bottom:1px solid var(--hairline);
  cursor:pointer;"
  onclick="toggleStrengthExercise('${dateKey}
  ', ${i})">
            <div
  style="width:20px;height:20px;border-
  radius:5px;border:1.5px solid
  ${checks[i]?'var(--brass)':'var(--hairline-
  brass)'}; background:${checks[i]?'var(--
  brass-soft)':'transparent'}; display:flex;
  align-items:center; justify-content:center;
  flex-shrink:0; color:var(--brass); font-
  size:13px;">
              ${checks[i] ? '✓' : ''}

```

```

        </div>
        <div style="font-size:14px; text-
decoration:${checks[i]?'line-
through':'none'}; color:${checks[i]?'var(--
lane-dim)':'var(--chalk)'};">${ex}</div>
        </div>`).join('')
    </div>`;
}

/* ===== RECOVERY CARD =====
*/
function renderRecoveryCard(dateKey,
recovery){
    return `
        <div class="card accent">
            <h3>Recovery – replenish now</h3>
            <p style="font-size:13px; color:var(-
-lane-dim); font-family:'Helvetica
Neue',Arial,sans-serif; margin:0 0 12px;">
                Estimated from
                ${formatDistance(recovery.miles)} at
                ~${recovery.minutes} minutes. For a sharper
                number based on what you actually burned,
                attach today's Garmin or Strava screenshot
                to the Coach and ask for a recovery check.
            </p>
            <div class="row">
                <div><div style="font-size:11px;
color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-
serif;">Carbs</div><div class="big-stat"
style="font-size:24px;">${recovery.carbsG}
<span class="unit">g</span></div></div>
                <div><div style="font-size:11px;

```

```
color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-
serif;">Protein</div><div class="big-stat"
style="font-
size:24px;">${recovery.proteinG}<span
class="unit">g</span></div></div>
</div>
<div class="row" style="margin-
top:12px;">
  <div><div style="font-size:11px;
color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-
serif;">Fluid</div><div class="big-stat"
style="font-
size:24px;">${(recovery.fluidMl/1000).toFixed(1)}<span class="unit">L</span></div>
</div>
  <div><div style="font-size:11px;
color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-
serif;">Sodium</div><div class="big-stat"
style="font-
size:24px;">${recovery.sodiumMg}<span
class="unit">mg</span></div></div>
</div>
  <p style="font-size:12.5px;
color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-serif;
line-height:1.6; margin-top:14px;">
    Best within the first 30-60
minutes: a carb+protein combo (recovery
shake, chocolate milk, or a proper meal
with both) plus steady fluid intake over
the next couple of hours rather than all at
```

```

once.
    </p>
    <button class="ghost block"
style="margin-top:12px;"
onclick="setView('coach')">Ask coach with a
screenshot instead</button>
    </div>`;
}

/* ===== PACE SUMMARY & WEEKLY
MILEAGE ===== */
/* ===== SESSION TOTALS & NAMING
===== */
function sessionTotalMiles(session){
    if(!session) return 0;
    if(session.segments &&
session.segments.length){
        let total = 0;
        session.segments.forEach(seg=>{
            if(seg.kind === 'distance') total +=
seg.miles || 0;
            else if(seg.kind === 'reps') total +=
(seg.reps||0) * (seg.repDistance||0);
        });
        return Math.round(total*10)/10;
    }
    return session.miles || 0;
}
function segmentMilesRecap(session){
    if(!session || !session.segments) return
null;
    const parts = session.segments.map(seg=>{
        if(seg.kind === 'distance') return
`${seg.miles}`;

```

```

        if(seg.kind === 'reps') return
        `${seg.reps}×${seg.repDistance}`;
        return null;
    }).filter(Boolean);
    return parts.length ? parts.join(' + ') :
    null;
}
function sessionTypeName(session){
    if(!session) return 'Session';
    if(session.type === 'race') return 'Race
day';
    if(session.badge === 'long') return 'Long
run';
    if(session.badge === 'rest') return 'Rest
day';
    if(session.badge === 'cross') return
'Cross-train';
    if(session.badge === 'strength') return
'Strength';
    const d =
(session.desc||'').toLowerCase();
    if(session.type === 'quality' ||
session.badge === 'quality'){
        if(d.includes('tempo')) return 'Tempo
run';
        if(/x\s*\d+(\.\d+)?\s*mile/.test(d) ||
d.includes('interval')) return 'Interval
workout';
        if(d.includes('marathon')) return
'Marathon-pace run';
        if(d.includes('stride')) return 'Easy +
strides';
        return 'Quality session';
    }
}

```



```

    if(session.badge === 'easy') return 'Easy
run';
    return 'Session';
}

function paceSummary(eff){
    if(!eff) return null;
    const total =
sessionTotalMiles(eff.session);
    if(!total) return null;
    const paces =
trainingPaces(getActiveRace());
    let paceText;
    if(!paces){
        paceText =
fmtPace(state.settings.easyPaceMinPerMile);
    } else if(eff.session.type === 'race'){
        paceText = fmtPace(paces.marathon);
    } else if(eff.session.type === 'quality')
{
        paceText = `${fmtPace(paces.tempo)}-
${fmtPace(paces.interval)}`;
    } else {
        paceText = fmtPace(paces.easy);
    }
    return `${formatDistance(total)} planned
· target pace ${paceText}`;
}

function weekMileageSummary(weekStartDate){
    const ws = weekStartDate ||
startOfWeek(new Date());
    const week = buildWeekSchedule(ws);
    let totalPlanned = 0, completed = 0;
    week.forEach(d=>{

```

```

    const ov = state.overrides[d.dateKey];
    const sess = ov || d.session;
    const miles = sess.miles || 0;
    totalPlanned += miles;
    if(state.log[d.dateKey] &&
state.log[d.dateKey].done) completed +=
miles;
  });
  return { totalPlanned, completed,
remaining: Math.max(totalPlanned-
completed,0) };
}
function renderWeekMileageCard(){
  const m = weekMileageSummary();
  const pct = m.totalPlanned > 0 ?
Math.min(100,
Math.round((m.completed/m.totalPlanned)*100
)) : 0;
  return `
    <div class="card">
      <h3>This week's mileage</h3>
      <div class="big-stat" style="font-
size:28px;">${formatDistance(m.completed)}
<span class="unit">of
${formatDistance(m.totalPlanned)}</span>
</div>
      <div class="macro-bar" style="margin-
top:10px;"><div class="fill"
style="width:${pct}%; background:#3aa0ff;">
</div></div>
      <p style="font-size:12.5px;
color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-serif;
margin-top:10px;">

```

```

        Expected by end of week:
        ${formatDistance(m.totalPlanned)}${m.remaining>0 ? ` · ${formatDistance(m.remaining)}
to go` : ` · week complete`}
      </p>
    </div>`;
  }

  /* ===== RUN BREAKDOWN =====
  */
  /* ===== SEGMENT DETAIL (exact pace
per quality-session segment) =====
  */
  const PACE_TYPE_LABELS = { easy:'easy',
tempo:'tempo', interval:'interval (5K-10K
effort)', marathon:'marathon goal' };
  const PACE_TYPE_ZONE = { easy:'z2',
tempo:'z3', interval:'z4', marathon:'z3' };
  const PACE_TYPE_FEEL = {
    easy: 'should feel conversational – you
could hold a chat the whole way.',
    tempo: 'comfortably hard – you could say
a short sentence, not hold a
conversation.',
    interval: 'hard – by the last rep this
should feel like real effort, not
comfortable.',
    marathon: 'controlled and sustainable –
this is the pace you want to feel automatic
by race day.'
  };
  function renderSegmentDetail(seg, paces,
zones){
    const zoneKey =

```

```

PACE_TYPE_ZONE[seg.paceType];
    const zoneText = (zones && zoneKey &&
zones[zoneKey]) ? ` (Zone
${zoneKey.slice(1)}:
${zones[zoneKey].lo}-${zones[zoneKey].hi}bp
m)` : '';
    if(seg.kind === 'distance'){
        const p = paces ? paces[seg.paceType] :
null;
        return `${formatDistance(seg.miles)} at
${PACE_TYPE_LABELS[seg.paceType]}|seg.paceT
ype} pace${p?`
(${fmtPace(p)})`:''}${zoneText} -
${PACE_TYPE_FEEL[seg.paceType]}|'|`;
    }
    if(seg.kind === 'reps'){
        const p = paces ? paces[seg.paceType] :
null;
        return `${seg.reps} x
${formatDistance(seg.repDistance)} at
${PACE_TYPE_LABELS[seg.paceType]}|seg.paceT
ype} pace${p?`
(${fmtPace(p)})`:''}${zoneText},
${seg.recovery} -
${PACE_TYPE_FEEL[seg.paceType]}|'|`;
    }
    return seg.customText;
}

function buildRunBreakdown(eff){
    if(!eff) return [];
    const session = eff.session;
    const type = session.type;
    const miles = sessionTotalMiles(session);

```

```

    const race = getActiveRace();
    const paces = trainingPaces(race) || {
    easy: state.settings.easyPaceMinPerMile,
    marathon:null, tempo:null, interval:null };
    const zones = hrZones();
    const zoneNote = zones ? ` Aim to keep
heart rate around Zone 2
(${zones.z2.lo}-${zones.z2.hi}bpm) for easy
running.` : '';

    if(type === 'long'){
    return [
        { label:'Warm-up',
detail:`${formatDistance(1)} easy jog to
ease in – well below marathon effort.` },
        { label:'Main long run',
detail:`${formatDistance(Math.max(miles-
1,0))} at easy-to-steady pace
(${fmtPace(paces.easy)}).${zoneNote} If
you're feeling strong, let the last few
miles drift toward marathon pace
(${fmtPace(paces.marathon)}).` },
        { label:'Fueling', detail:`Carry
water or an electrolyte drink. Once you're
past 60-75 minutes, take a gel or ~30g
carbs every 40-45 minutes to practise race-
day fueling.` },
        { label:'Cool-down', detail:`5-10
minutes walking, then stretch calves,
hamstrings, and hip flexors.` }
    ];
    }
    if(type === 'quality'){
    if(session.segments &&

```

```

session.segments.length){
    return session.segments.map(seg => ({
label: seg.label, detail:
renderSegmentDetail(seg, paces, zones) }));
    }
    // fallback for manually swapped
sessions that don't carry structured
segments
    return [
        { label:'Warm-up',
detail:`${formatDistance(2)} easy plus 4 x
20sec relaxed strides to open the legs up.`
},
        { label:'Main set', detail:
convertDistanceWording(session.desc) },
        { label:'Cool-down',
detail:`${formatDistance(1)} easy jog to
bring the heart rate down gradually.` }
    ];
}
if(type === 'easy'){
    return [
        { label:'Whole run',
detail:`${formatDistance(miles)} at easy,
conversational pace
(${fmtPace(paces.easy)}).${zoneNote}` }
    ];
}
if(type === 'cross'){
    return [
        { label:'Warm-up', detail:`5 minutes
easy spin or elliptical.` },
        { label:'Main effort', detail:`~30
minutes at a steady, moderate effort -

```

```

should feel about like an easy run, not a
hard workout.` },
    { label:'Cool-down', detail:`5
minutes easy, then a light stretch.` }
  ];
}
if(type === 'race'){
  return [
    { label:'Goal pace',
detail:`${fmtPace(paces.marathon)}
average.` },
    { label:'First 10K', detail:`Hold
back – run 10-15sec/mi slower than goal
pace. Adrenaline makes this feel too easy;
it isn't.` },
    { label:'Middle miles',
detail:`Settle into goal pace once you're
warmed up and the field has spread out.` },
    { label:'Final 10K', detail:`This is
where the race is won or lost – hold pace
as long as you can, and it's fine to slow
slightly rather than blow up.` },
    { label:'Fueling', detail:`~30-60g
carbs per hour from gels/drinks, starting
around 45 minutes in, and don't skip aid
stations.` }
  ];
}
return [{ label:'Rest day', detail:`No
structured training. Prioritise sleep,
hydration, and light movement like walking
if you feel like it.` }];
}

```

```

/* ===== AI MEAL IDEAS =====
*/
async function fetchMealIdeas(){
  if(!state.settings.apiKey){ alert('Add
your Anthropic API key in Settings first –
meal ideas use the same coach
connection.');
```

 return; }
 const key = todayKey();
 window._mealIdeasLoading = true;
 render();
 const n = nutritionForDay(key);
 const prompt = `Suggest practical, real
whole-food options – not a rigid meal plan
– to hit today's targets: ~\${n.totalKcal}
kcal, \${n.carbsG}g carbs, \${n.proteinG}g
protein, \${n.fatG}g
fat\${n.needsElectrolytes ? ' plus
electrolytes' : ''}. For each of protein,
carbs, and fat give 3-4 specific common
food options with roughly how much of that
food gets them a meaningful chunk of the
target (e.g. name a food, say whether it's
a strong source, and roughly how much to
eat). Keep it practical, non-preachy, and
skip the disclaimers – just the food
suggestions.`;
 try{
 const resp = await
 fetch('https://api.anthropic.com/v1/message
s', {
 method: 'POST',
 headers: {
 'Content-Type': 'application/json',
 'x-api-key': state.settings.apiKey,


```

        'anthropic-version': '2023-06-01',
        'anthropic-dangerous-direct-
browser-access': 'true'
    },
    body: JSON.stringify({ model:'claude-
sonnet-4-6', max_tokens:700, system:
buildCoachSystemPrompt(), messages:
[{'role':'user', content: prompt}] })
    });
    const data = await resp.json();
    const textBlock = (data.content||
[]).find(b=>b.type==='text');
    state.mealIdeas[key] = textBlock ?
textBlock.text : "Couldn't get suggestions
right now – try again in a moment.";
    }catch(e){
        state.mealIdeas[key] = `Couldn't reach
the API: ${e.message}`;
    }
    window._mealIdeasLoading = false;
    saveState();
    render();
}

/* ===== BARCODE FOOD LOGGING
===== */
async function lookupBarcode(){
    const barcode =
document.getElementById('barcodeInput').val
ue.trim();
    if(!barcode){ showToast('Enter a barcode
number first'); return; }
    window._barcodeLoading = true;
    window._barcodeError = null;

```

```

window._barcodePreview = null;
render();
try{
    const resp = await
fetch(`https://world.openfoodfacts.org/api/
v2/product/${encodeURIComponent(barcode)}.j
son`);
    const data = await resp.json();
    if(data.status !== 1 || !data.product){
        window._barcodeError = 'No product
found for that barcode.';
    } else {
        const p = data.product;
        const n = p.nutriments || {};
        window._barcodePreview = {
            barcode,
            name: p.product_name ||
p.generic_name || 'Unknown product',
            per100: {
                kcal: Math.round(n['energy-
kcal_100g'] || 0),
                protein:
Math.round((n.proteins_100g||0)*10)/10,
                carbs:
Math.round((n.carbohydrates_100g||0)*10)/10
            ,
                fat:
Math.round((n.fat_100g||0)*10)/10
            }
        };
    }
}catch(e){
    window._barcodeError = `Lookup failed:
${e.message}`;
}

```

```

    }
    window._barcodeLoading = false;
    render();
  }
  async function onBarcodePhotoChosen(e){
    const file = e.target.files[0];
    if(!file) return;
    if(!('BarcodeDetector' in window)){
      showToast("Auto-read isn't supported on
this browser – type the number below
instead");
      return;
    }
    try{
      const bitmap = await
createImageBitmap(file);
      const detector = new
BarcodeDetector({formats:
['ean_13','ean_8','upc_a','upc_e']});
      const codes = await
detector.detect(bitmap);
      if(codes && codes[0]){

document.getElementById('barcodeInput').val
ue = codes[0].rawValue;
      lookupBarcode();
    } else {
      showToast("Couldn't read a barcode in
that photo – try again or type it in");
    }
  }catch(err){
    showToast('Barcode reading failed –
type the number in instead');
  }
}

```

```

}
function addFoodLogEntry(){
  const preview = window._barcodePreview;
  if(!preview) return;
  const grams =
Number(document.getElementById('barcodeGram
s').value) || 100;
  const factor = grams/100;
  const key = todayKey();
  if(!state.foodLog[key])
state.foodLog[key] = [];
  state.foodLog[key].push({
    barcode: preview.barcode, name:
preview.name, grams,
    kcal:
Math.round(preview.per100.kcal*factor),
    protein:
Math.round(preview.per100.protein*factor*10
)/10,
    carbs:
Math.round(preview.per100.carbs*factor*10)/
10,
    fat:
Math.round(preview.per100.fat*factor*10)/10
,
    loggedAt: Date.now()
  });
  saveState();
  window._barcodePreview = null;
  showToast(`Added ${preview.name}`);
  render();
}
function removeFoodLogEntry(dateKey, idx){
  state.foodLog[dateKey].splice(idx,1);

```

```

    saveState();
    render();
  }
  function dailyLoggedTotals(dateKey){
    const entries = (state.foodLog &&
state.foodLog[dateKey]) || [];
    return entries.reduce((acc,e)=>({
      kcal:acc.kcal+e.kcal,
protein:acc.protein+e.protein,
carbs:acc.carbs+e.carbs, fat:acc.fat+e.fat
    }), {kcal:0,protein:0,carbs:0,fat:0});
  }

  function showToast(msg){
    const t =
document.getElementById('toast');
    t.textContent = msg;
    t.classList.add('show');

    setTimeout(()=>t.classList.remove('show'),
2200);
  }

  /* ===== ROUTER ===== */
  let currentView = 'today';
  function setView(v){
    currentView = v;
    document.querySelectorAll('nav.tabbar
button').forEach(b=>{
      b.classList.toggle('active',
b.dataset.view === v);
    });
    render();
  }

```

```
document.querySelectorAll('nav.tabbar
button').forEach(b=>{
    b.addEventListener('click',
    ()=>setView(b.dataset.view));
});

function render(){
    const main =
document.getElementById('main');
    main.innerHTML = '';
    const section =
document.createElement('section');
    section.className = 'view active';
    if(currentView === 'today')
section.innerHTML = renderToday();
    else if(currentView === 'plan')
section.innerHTML = renderPlan();
    else if(currentView === 'fuel')
section.innerHTML = renderFuel();
    else if(currentView === 'strength')
section.innerHTML = renderStrength();
    else if(currentView === 'daydetail')
section.innerHTML =
renderDayDetail(window._selectedDateKey);
    else if(currentView === 'coach')
section.innerHTML = renderCoach();
    else if(currentView === 'shifts')
section.innerHTML = renderShifts();
    else if(currentView === 'settings')
section.innerHTML = renderSettings();
    main.appendChild(section);
    attachViewHandlers();
    updateHeader();
}
```

```

function updateHeader(){
  const el =
document.getElementById('raceCountdown');
  const race = getActiveRace();
  const raceDate = parseKey(race.date);
  const today = new Date();
today.setHours(0,0,0,0);
  const days = daysBetween(today,
raceDate);
  if(days < 0){ el.innerHTML =
`<strong>Race day has passed</strong>`;
return; }
  el.innerHTML = `<strong>${days}</strong>
days to go<br>${race.name}`;
}

/* ===== TODAY VIEW ===== */
function renderToday(){
  const key = todayKey();
  const eff = getEffectiveSession(key);
  const nutrition = nutritionForDay(key);
  const checkin = state.checkins[key] ||
{};
  const shift = getShift(key);

  const badge = eff ? eff.session.badge :
'rest';
  const desc = eff ?
convertDistanceWording(eff.session.desc) :
'No plan generated yet';
  const done = state.log[key] &&
state.log[key].done;

```

```

    // month strip – swipeable across the
    whole current month, centered on today
    const now = new Date();
    const monthStart = new
Date(now.getFullYear(), now.getMonth(), 1);
    const monthEnd = new
Date(now.getFullYear(), now.getMonth()+1,
0);
    const stripDates = [];
    for(let d = new Date(monthStart); d <=
monthEnd; d = addDays(d,1))
stripDates.push(new Date(d));

    const weekStripHtml = stripDates.map(d=>{
    const dk = fmtDate(d);
    const isToday = dk === key;
    const done = isDayDone(dk);
    return `<div class="day-chip
${isToday?'today':''} ${done?'done-
day':''}" data-goto-day="${dk}"
${isToday?'id="todayChip"':''}>
        <div class="dow">${dowShort(d)}</div>
        <div class="num">${d.getDate()}</div>
        ${dayDots(dk)}
    </div>`;
    }).join('');

    return `
    <h1 class="page-title">Today${eff &&
eff.phase ? ` · ${eff.phase}` : ''}</h1>
    <p class="page-sub">${monthDayLabel(new
Date())} – ${shift !== 'unknown' ?
shiftLabelWithHours(shift) : 'Shift not set
for today'}</p>

```



```

<div class="week-
strip">${weekStripHtml}</div>

<div class="card accent">
  <div style="display:flex; justify-
content:space-between; align-items:flex-
start; margin-bottom:8px;">
    <h3 style="margin:0;">Today's
session</h3>
    ${eff ? `<div style="text-
align:right;">
      <div style="font-
family:'Helvetica Neue',Arial,sans-serif;
font-weight:700; font-size:13px;
color:var(--
gold);">${sessionTypeName(eff.session)}
</div>
      ${sessionTotalMiles(eff.session)
? `<div style="font-family:'Helvetica
Neue',Arial,sans-serif; font-size:12px;
color:var(--lane-
dim);">${formatDistance(sessionTotalMiles(e
ff.session))}
total${segmentMilesRecap(eff.session) ? `
(${segmentMilesRecap(eff.session)})` : ''}
</div>` : ''}
      </div>` : ''}
    </div>
  <div style="display:flex; align-
items:flex-start; gap:10px; margin-
bottom:6px;">
    <span class="badge
${badge}">${badge}</span>

```

```

    </div>
    <div style="font-size:17px; margin:
6px 0 4px;">${desc}</div>
    ${paceSummary(eff) ? `<div
style="font-size:13px; color:var(--gold);
font-family:'Helvetica Neue',Arial,sans-
serif; margin-top:4px;">${paceSummary(eff)}
</div>` : ''}
    ${eff && eff.session.adjusted ? `<div
style="font-size:12px; color:var(--lane-
dim); margin-top:6px;">Originally planned:
${eff.session.original}</div>` : ''}
    <div style="margin-top:14px;
display:flex; gap:10px; flex-wrap:wrap;">
      <button
onclick="toggleDone('${key}')"
class="${done?'ghost':''}">${done ? '✓
Marked done' : 'Mark as done'}</button>
      <button class="ghost"
onclick="openOverride('${key}')">Swap
session</button>
      <button class="ghost"
onclick="toggleBreakdown()">${window._today
BreakdownOpen ? 'Hide' : 'View'} full
breakdown</button>
    </div>
  </div>

  ${renderWeekMileageCard()}

  ${window._todayBreakdownOpen ? `
<div class="card">
  <h3>Run breakdown</h3>
  ${buildRunBreakdown(eff).map(seg=>`

```

```

        <div style="margin-bottom:14px;">
            <div style="font-
family:'Helvetica Neue',Arial,sans-serif;
font-size:12px; color:var(--gold); font-
weight:600; margin-
bottom:3px;">${seg.label}</div>
            <div style="font-size:14px; line-
height:1.55;">${seg.detail}</div>
        </div>`).join('')}
    </div>` : ''}

    ${eff && eff.session.strength ? `
    <div class="card">
        <h3><span class="badge strength"
style="margin-
right:8px;">strength</span>${eff.session.st
rength.label}</h3>
        ${renderStrengthChecklist(key,
eff.session.strength)}
    </div>` : ''}

    <div class="card">
        <h3>How are you feeling?</h3>
        <label>Energy</label>
        <div class="mood-
row">${[1,2,3,4,5].map(n=>`<div
class="mood-btn
${checkin.energy===n?'selected':''}" data-
checkin="energy" data-val="${n}">${n}
</div>`).join('')}</div>
        <label>Soreness</label>
        <div class="mood-
row">${[1,2,3,4,5].map(n=>`<div
class="mood-btn

```

```

    ${checkin.soreness===n?'selected':''}"
    data-checkin="soreness" data-
    val="${n}">${n}</div>`).join('')</div>
    <label>Mood</label>
    <div class="mood-
    row">${['😞','😓','😐','😌','😄'].map((e,i)
    =>`<div class="mood-btn
    ${checkin.mood===i+1?'selected':''}" data-
    checkin="mood" data-val="${i+1}">${e}
    </div>`).join('')</div>
    <textarea id="checkinNote"
    placeholder="Anything the coach should know
    today? (sore knee, bad sleep, stressful
    shift...)">${checkin.note||''}</textarea>
    <button class="block"
    onclick="saveCheckin('${key}')">Save check-
    in</button>
    </div>

```

```

    ${window._coachNoteLoading === key ?
    `<div class="card"><p style="font-
    size:13px; color:var(--lane-dim); font-
    family:'Helvetica Neue',Arial,sans-serif;
    margin:0;">Coach is looking at your check-
    in...</p></div>` : ''}
    ${checkin.coachNote ? `<div class="card
    accent"><h3>Coach note</h3><p style="font-
    size:14px; line-height:1.6;
    margin:0;">${escapeHtml(checkin.coachNote)}
    </p></div>` : ''}

```

```

<div class="card">
  <h3>Today's fuel targets</h3>
  <div class="row">

```

```

        <div><div style="font-size:11px;
color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-
serif;">Calories</div><div class="big-stat"
style="font-
size:28px;">${nutrition.totalKcal}</div>
</div>

        <div><div style="font-size:11px;
color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-
serif;">Water</div><div class="big-stat"
style="font-
size:28px;">${(nutrition.waterMl/1000).toFi
xed(1)}<span class="unit">L</span></div>
</div>

        </div>
        ${nutrition.needsElectrolytes ? `<div
style="margin-top:10px; font-size:13px;
color:var(--gold); font-family:'Helvetica
Neue',Arial,sans-serif;">⚡ Add
electrolytes – today's session is long/hard
enough to need them</div>` : ''}
        <button class="ghost block"
style="margin-top:12px;"
onclick="setView('fuel')">See full
breakdown</button>
        </div>

        ${done && recoveryForSession(key) ?
renderRecoveryCard(key,
recoveryForSession(key)) : ''}
        `;
    }
}

```

```
function toggleDone(key){
  if(!state.log[key]) state.log[key] = {};
  state.log[key].done =
!state.log[key].done;
  saveState();
  render();
}
function toggleBreakdown(){
  window._todayBreakdownOpen =
!window._todayBreakdownOpen;
  render();
}

document.addEventListener('click', (e)=>{
  const moodBtn = e.target.closest('.mood-
btn');
  if(moodBtn){
    const key = todayKey();
    if(!state.checkins[key])
state.checkins[key] = {};
    state.checkins[key]
[moodBtn.dataset.checkin] =
Number(moodBtn.dataset.val);
    saveState();
    render();
    return;
  }
  const dayChip = e.target.closest('[data-
goto-day]');
  if(dayChip){
    openDayDetail(dayChip.dataset.gotoDay);
  }
});
```

```

async function saveCheckin(key){
  if(!state.checkins[key])
state.checkins[key] = {};
  state.checkins[key].note =
document.getElementById('checkinNote').value;
  saveState();
  showToast('Check-in saved');
  render();

  const c = state.checkins[key];
  const concerning = (c.energy &&
c.energy<=2) || (c.soreness &&
c.soreness>=4) || (c.mood && c.mood<=2);
  if(concerning && state.settings.apiKey){
    await fetchCoachCheckinNote(key);
  }
}
async function fetchCoachCheckinNote(key){
  window._coachNoteLoading = key;
  render();
  const c = state.checkins[key] || {};
  const eff = getEffectiveSession(key);
  const prompt = `The athlete just logged
today's check-in: energy ${c.energy||'-'}/5, soreness ${c.soreness||'-'}/5, mood
${c.mood||'-'}/5${c.note?'`, note:
"${c.note}"`:`'`. Their plan for today
(already auto-adjusted for how they're
feeling if applicable) is: ${eff ?
eff.session.desc : 'not generated'}. Give a
brief, warm coach note (2-4 sentences):
acknowledge how they're feeling, confirm
what today should actually look like given

```

```

that, and one quick, concrete tip relevant
right now (nutrition, hydration, sleep, or
recovery). No disclaimers, no generic
filler.`;
    try{
        const resp = await
fetch('https://api.anthropic.com/v1/message
s', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
            'x-api-key': state.settings.apiKey,
            'anthropic-version': '2023-06-01',
            'anthropic-dangerous-direct-
browser-access': 'true'
        },
        body: JSON.stringify({ model:'claude-
sonnet-4-6', max_tokens:300, system:
buildCoachSystemPrompt(), messages:
[{'role':'user', content: prompt}] })
    });
    const data = await resp.json();
    const textBlock = (data.content||
[]).find(b=>b.type==='text');
    if(textBlock){
        state.checkins[key].coachNote =
textBlock.text;
        state.chat.push({role:'assistant',
content: textBlock.text, ts:Date.now()});
        saveState();
        speakText(textBlock.text);
    }
}catch(e){ /* non-critical, fail silently
*/ }

```



```

    window._coachNoteLoading = null;
    render();
  }

  function openOverride(key){
    const desc = prompt('Describe the session
you want instead (e.g. "Rest – travelling
today")');
    if(desc === null) return;
    const badge = prompt('Category: long /
quality / easy / cross / rest', 'easy') ||
'easy';
    state.overrides[key] = { type: badge,
badge: badge, desc: desc, miles: 0 };
    saveState();
    render();
  }

  function openDayDetail(dateKey){
    window._previousView = currentView;
    window._selectedDateKey = dateKey;
    window._dayDetailBreakdownOpen = false;
    currentView = 'daydetail';
    render();
  }
  function closeDayDetail(){
    currentView = window._previousView ||
'plan';
    render();
  }
  function toggleDayDetailBreakdown(){
    window._dayDetailBreakdownOpen =
!window._dayDetailBreakdownOpen;
    render();
  }

```

```

}
function renderDayDetail(dateKey){
  if(!dateKey) return `
```

```

        <h3 style="margin:0;">Session</h3>
        ${eff ? `<div style="text-align:right;">
            <div style="font-family:'Helvetica Neue',Arial,sans-serif;
font-weight:700; font-size:13px;
color:var(--gold);">${sessionTypeName(eff.session)}
</div>
                ${sessionTotalMiles(eff.session)
? `<div style="font-family:'Helvetica
Neue',Arial,sans-serif; font-size:12px;
color:var(--lane-dim);">${formatDistance(sessionTotalMiles(e
ff.session))}
total${segmentMilesRecap(eff.session) ? `
(${segmentMilesRecap(eff.session))}` : ''}
</div>` : ''}
                </div>` : ''}
            </div>
            <span class="badge ${badge}">${badge}
</span>
            <div style="font-size:17px;
margin:8px 0 4px;">${desc}</div>
            ${paceSummary(eff) ? `<div
style="font-size:13px; color:var(--gold);
font-family:'Helvetica Neue',Arial,sans-
serif; margin-top:4px;">${paceSummary(eff)}
</div>` : ''}
            ${eff && eff.session.adjusted ? `<div
style="font-size:12px; color:var(--lane-
dim); margin-top:6px;">Originally planned:
${eff.session.original}</div>` : ''}
            <div style="margin-top:14px;

```

```

display:flex; gap:10px; flex-wrap:wrap;">
    <button
onclick="toggleDone('${dateKey}')"
class="${done?'ghost':''}">${done ? '✓
Marked done' : 'Mark as done'}</button>
    <button class="ghost"
onclick="openOverride('${dateKey}')">Swap
session</button>
    <button class="ghost"
onclick="toggleDayDetailBreakdown()">${wind
ow._dayDetailBreakdownOpen ? 'Hide' :
'View'} full breakdown</button>
</div>
</div>

${window._dayDetailBreakdownOpen ? `
<div class="card">
    <h3>Run breakdown</h3>
    ${buildRunBreakdown(eff).map(seg=>`
        <div style="margin-bottom:14px;">
            <div style="font-
family:'Helvetica Neue',Arial,sans-serif;
font-size:12px; color:var(--gold); font-
weight:600; margin-
bottom:3px;">${seg.label}</div>
            <div style="font-size:14px; line-
height:1.55;">${seg.detail}</div>
        </div>`).join('')}
    </div>` : ''}

${eff && eff.session.strength ? `
<div class="card">
    <h3>${eff.session.strength.label}
</h3>

```

```

        ${renderStrengthChecklist(dateKey,
eff.session.strength)}
    </div>` : ''}

    <div class="card">
        <h3>Fuel & fluids for this
day</h3>
        <div class="row">
            <div><div style="font-size:11px;
color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-
serif;">Calories</div><div class="big-stat"
style="font-
size:26px;">${nutrition.totalKcal}</div>
</div>
            <div><div style="font-size:11px;
color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-
serif;">Water</div><div class="big-stat"
style="font-
size:26px;">${(nutrition.waterMl/1000).toFi
xed(1)}<span class="unit">L</span></div>
</div>
            <div style="margin-top:12px; font-
size:13px; font-family:'Helvetica
Neue',Arial,sans-serif; color:var(--lane-
dim);">
                Carbs ${nutrition.carbsG}g ·
Protein ${nutrition.proteinG}g · Fat
${nutrition.fatG}g${nutrition.needsElectrol
ytes ? ' · + electrolytes' : ''}
            </div>
        </div>
    </div>

```

```

    ${done && recovery ?
renderRecoveryCard(dateKey, recovery) : ''}
    `;
}

/* ===== PLAN VIEW ===== */
function renderPlan(){
    const today = new Date();
    const ws = startOfWeek(today);
    window._planWeekOffset =
window._planWeekOffset || 0;
    const viewWs = addDays(ws,
window._planWeekOffset*7);
    const week = buildWeekSchedule(viewWs);
    const phase = week[0].phase;
    const weeksOut = week[0].weeksOut;

    const rows = week.map(day=>{
        const key = day.dateKey;
        const effOverride =
state.overrides[key];
        const s = effOverride || day.session;
        const isToday = key === todayKey();
        const done = state.log[key] &&
state.log[key].done;
        return `<div class="session-item
${done?'done':''}" style="cursor:pointer;"
onclick="openDayDetail('${key}')">
            <div class="s-
day">${dowShort(day.date)}
${day.date.getDate()}${isToday?' •':''}
            </div>
            <div class="s-desc">

```

```

        ${convertDistanceWording(s.desc)}

<small>${SHIFT_LABELS[day.shift]}${sessionTotalMiles(s) ? ` ·
${formatDistance(sessionTotalMiles(s))}` :
''}${day.session.strength ? ' · + strength'
: ''}</small>
    </div>
    <span class="badge
${s.badge}">${s.badge}</span>
</div>`;
    }).join('');

    const race = getActiveRace();
    const realism = raceRealism(race);
    const paces = trainingPaces(race);
    const goalSet = goalTotalMinutes(race) !==
    null;

    const otherRaces =
    state.races.filter(r=>r.id !== race.id);

    return `
    <h1 class="page-title">Training
    plan</h1>
    <p class="page-sub">${race.name} ·
    ${weeksOut <= 0 ? 'Race week' :
    `${weeksOut} weeks out`} · ${phase}</p>

    ${otherRaces.length ? `<div
    class="pill-row">
    ${state.races.map(r=>`<div
    class="pill
    ${r.id===race.id?'selected':''}`

```

```
onclick="setActiveRace('${r.id}')">${r.name}
}${r.done?' ✓':''}</div>`).join('')}
</div>` : ''}
```

```
<div class="card ${goalSet &&
realism.verdict!=='unknown' &&
realism.verdict!=='no-goal' ? 'accent' :
''}">
    <h3>Goal & realism check</h3>
    ${goalSet ? `<div class="big-stat"
style="font-size:30px; margin-
bottom:8px;">${fmtMinutesAsHM(goalTotalMinu
tes(race))}</div>` : `<p style="font-
size:13px; color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-
serif;">No goal time set yet.</p>`}
    <p style="font-size:13px; line-
height:1.6; font-family:'Helvetica
Neue',Arial,sans-serif;
color:${realism.verdict==='unrealistic'?'#f
f6b6b':'var(--parchment)'}"; margin:0 0
12px;">${realism.text}</p>
    ${paces ? `<div class="row">
    <div><div style="font-size:11px;
color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-
serif;">Easy</div><div style="font-
weight:700;">${fmtPace(paces.easy)}</div>
</div>
    <div><div style="font-size:11px;
color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-
serif;">Marathon</div><div style="font-
weight:700; color:var(--
```



```

gold);">${fmtPace(paces.marathon)}</div>
</div>

    <div><div style="font-size:11px;
color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-
serif;">Tempo</div><div style="font-
weight:700;">${fmtPace(paces.tempo)}</div>
</div>

    <div><div style="font-size:11px;
color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-
serif;">Interval</div><div style="font-
weight:700;">${fmtPace(paces.interval)}
</div></div>
    </div>` : ''}
    <button class="ghost block"
style="margin-top:14px;"
onclick="setView('settings')">Set goal time
in Settings</button>
    </div>

    <div class="row" style="margin-
bottom:16px;">
        <button class="ghost"
onclick="shiftPlanWeek(-1)">← Prev
week</button>
        <button class="ghost"
onclick="shiftPlanWeek(1)">Next week
→</button>
    </div>

    <div class="card">
        <h3>${monthDayLabel(viewWs)} -
${monthDayLabel(addDays(viewWs,6))}</h3>

```

```

    ${rows}
  </div>

  <div class="card">
    <h3>About this plan</h3>
    <p style="font-size:13px; color:var(-
-lane-dim); font-family:'Helvetica
Neue',Arial,sans-serif; line-height:1.6;
margin:0;">
      Sessions are placed automatically
      around the shifts you've entered – long
      runs and quality sessions go on your best
      off/day-shift days, night-shift days
      default to rest. Tap "Swap session" on
      Today to override any day manually, or ask
      the Coach to restructure a week for you.
    </p>
  </div>
  `;
}
function shiftPlanWeek(n){
  window._planWeekOffset =
(window._planWeekOffset||0) + n;
  render();
}

/* ===== FUEL VIEW ===== */
function renderFuel(){
  const key = todayKey();
  const n = nutritionForDay(key);
  const w = getWeightKg();

  return `
    <h1 class="page-title">Fuel</h1>

```

```
<p class="page-sub">Based on  
${getWeightLb().toFixed(0)}lb bodyweight  
and today's ${n.type} session${n.miles?`  
(${formatDistance(n.miles)})`:''}</p>
```

```
<div class="card accent">  
  <h3>Calories</h3>  
  <div class="big-stat">${n.totalKcal}  
<span class="unit">kcal</span></div>  
  <p style="font-size:12px; color:var(-  
-lane-dim); font-family:'Helvetica  
Neue',Arial,sans-serif; margin-top:8px;">  
    ${n.baseTDEE} baseline +  
    ${n.runKcal} for today's running  
  </p>  
</div>
```

```
<div class="card">  
  <h3>Macros</h3>  
  <div class="macro-row">  
    <div class="label-line">  
<span>Carbs</span><span>${n.carbsG}g</span>  
</div>  
    <div class="macro-bar"><div  
class="fill" style="width:${Math.min(100,  
n.carbsG/6)}%; background:#3aa0ff;"></div>  
</div>  
    <div class="macro-row">  
      <div class="label-line">  
<span>Protein</span>  
<span>${n.proteinG}g</span></div>  
      <div class="macro-bar"><div  
class="fill" style="width:${Math.min(100,
```

```

n.proteinG/2.5)}}%; background:#22c55e;">
</div></div>
    </div>
    <div class="macro-row">
      <div class="label-line">
<span>Fat</span><span>${n.fatG}g</span>
</div>
      <div class="macro-bar"><div
class="fill" style="width:${Math.min(100,
n.fatG/1.5)}}%; background:#ff4d4d;"></div>
</div>
    </div>
  </div>

  <div class="card">
    <h3>Meal ideas for today's
targets</h3>
    ${window._mealIdeasLoading ? `<p
style="font-size:13px; color:var(--lane-
dim); font-family:'Helvetica
Neue',Arial,sans-serif;">Thinking of good
options...</p>` :
    state.mealIdeas[key] ? `<div
style="font-size:14px; line-height:1.7;
white-space:pre-
wrap;">${escapeHtml(state.mealIdeas[key])}
</div><button class="ghost block"
style="margin-top:12px;"
onclick="fetchMealIdeas()">Refresh
ideas</button>` :
    `<p style="font-size:13px;
color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-serif;
margin:0 0 12px;">Ask the coach for

```

```
specific real-food options – e.g. is tuna a
good protein source and how much would hit
today's number.</p><button class="block"
onclick="fetchMealIdeas()">Get meal
ideas</button>`}
</div>
```

```
<div class="card">
  <h3>Hydration</h3>
  <div class="big-stat" style="font-
size:32px;">${(n.waterMl/1000).toFixed(1)}
<span class="unit">litres today</span>
</div>
  <p style="font-size:13px; color:var(-
-lane-dim); font-family:'Helvetica
Neue',Arial,sans-serif; margin-top:10px;">
    ${n.minutes>0 ? `Includes extra for
~${n.minutes} min of running.` : 'Baseline
hydration for a rest/low-load day.'}
    ${n.needsElectrolytes ? ' Add an
electrolyte drink or tablet during/after –
today\'s session is long enough to need
it.' : ''}
  </p>
</div>
```

```
${renderFoodTracker(key, n)}
```

```
<div class="card">
  <h3>General guidance</h3>
  <p style="font-size:13px; color:var(-
-lane-dim); font-family:'Helvetica
Neue',Arial,sans-serif; line-height:1.7;
margin:0;">
```

On long-run and quality days, weight carbs toward before/during/after the session. On night shifts, keep meals lighter before sleep and prioritise protein + carbs at your "breakfast" after waking, whatever time that is. These are estimates, not medical advice – adjust based on how your weight and energy trend over a few weeks.

```
    </p>
  </div>
  `;
}

function renderFoodTracker(key, n){
  const totals = dailyLoggedTotals(key);
  const entries = state.foodLog[key] || [];
  const preview = window._barcodePreview;
  const canScanPhoto = true; // input
  always shown; auto-read only fires if
  browser supports BarcodeDetector

  return `
    <div class="card">
      <h3>Track what you eat</h3>
      <p style="font-size:12px; color:var(-
-lane-dim); font-family:'Helvetica
Neue',Arial,sans-serif; margin:0 0 12px;">
        Scan or type a barcode to look it
up (via Open Food Facts) and log it against
today's targets.
      </p>
      <div class="row" style="margin-
bottom:10px;">
```

```

        <button class="ghost"
onclick="document.getElementById('barcodePhotoInput').click()"><img alt="camera icon" data-bbox="485 228 505 243"/> Photo of
barcode</button>
        <input type="file"
id="barcodePhotoInput" accept="image/*"
capture="environment" style="display:none"
onchange="onBarcodePhotoChosen(event)">
    </div>
    <div class="row">
        <div style="flex:2;"><input
id="barcodeInput" placeholder="Or type
barcode number"></div>
        <div style="flex:1;"><button
class="block"
onclick="lookupBarcode()">${window._barcode
Loading ? '...' : 'Look up'}</button></div>
    </div>
    ${window._barcodeError ? `<p
style="font-size:13px; color:#ff8a80; font-
family:'Helvetica Neue',Arial,sans-
serif;">${window._barcodeError}</p>` : ''}

    ${preview ? `
        <div style="background:var(--ink-
field); border-radius:var(--radius-sm);
padding:14px; margin-top:6px;">
            <div style="font-weight:600;
margin-bottom:6px;">${preview.name}</div>
            <div style="font-size:12px;
color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-serif;
margin-bottom:10px;">
                Per 100g:

```

```

    ${preview.per100.kcal}kcal ·
    C${preview.per100.carbs}g ·
    P${preview.per100.protein}g ·
    F${preview.per100.fat}g
      </div>
      <label>Amount eaten (grams)
</label>
      <input id="barcodeGrams"
type="number" value="100">
      <button class="block"
onclick="addFoodLogEntry()">Add to today's
log</button>
    </div>
    ` : '`

    ${entries.length ? `
      <div style="margin-top:16px;">
        ${entries.map((e,i)=>`
          <div style="display:flex;
justify-content:space-between; align-
items:center; padding:8px 0; border-
bottom:1px solid var(--hairline);">
            <div style="font-
size:13px;">${e.name} <span
style="color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-serif;
font-size:11px;">(${e.grams}g)</span></div>
            <div style="display:flex;
align-items:center; gap:10px;">
              <span style="font-
size:12px; color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-
serif;">${e.kcal}kcal</span>
              <button class="ghost"

```



```

style="padding:4px 9px; font-size:11px;"
onclick="removeFoodLogEntry('${key}',
${i})">X</button>
    </div>
    </div>`).join('')}
</div>
<div class="row" style="margin-
top:14px;">
    <div><div style="font-size:11px;
color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-
serif;">Logged</div><div style="font-
weight:700;">${totals.kcal} /
${n.totalKcal} kcal</div></div>
    <div><div style="font-size:11px;
color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-
serif;">Protein</div><div style="font-
weight:700;">${totals.protein.toFixed(0)} /
${n.proteinG}g</div></div>
    </div>
    ` : `<p style="font-size:12px;
color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-serif;
margin-top:14px;">Nothing logged yet today.
</p>`
    </div>
    `;
}

/* ===== STRENGTH VIEW =====
*/
const STRENGTH_PHASE_NOTES = {
    build: "You're building a strength base:

```

two sessions a week of compound lifts and single-leg stability work. This is when to add load and build real durability – hips, glutes, and calves are what keep you upright at mile 20.",

peak: "Volume drops so your legs stay fresh for the big long runs, but intensity stays – one focused session a week keeps what you've built without adding fatigue.",

taper: "Light activation only from here. The goal is to keep muscles firing, not to build anything new – nothing that leaves you sore before race day.",

race: "No loaded strength work race week. Just enough mobility to stay loose."

};

function renderStrength(){

 window._strengthWeekOffset =

window._strengthWeekOffset || 0;

 const ws = startOfWeek(new Date());

 const viewWs = addDays(ws,

window._strengthWeekOffset*7);

 const week = buildWeekSchedule(viewWs);

 const weeksOut = week[0].weeksOut;

 const tpl =

templateForWeeksOut(weeksOut);

 const phaseNote =

STRENGTH_PHASE_NOTES[tpl.strengthBlock] ||

STRENGTH_PHASE_NOTES.build;

 const strengthDays = week.filter(d =>
d.session.strength);

 const dayCards = strengthDays.map(d=>{

```

    const isToday = d.dateKey ===
todayKey();
    return `
```

```
margin:0;">${phaseNote}</p>
</div>
```

```
    ${dayCards || ``<div class="card"><div
class="empty">No strength sessions
scheduled this week – race week or a rest-
heavy stretch.</div></div>`}

```

```
    <div class="card">
      <h3>General principles</h3>
      <p style="font-size:13px; color:var(-
-lane-dim); font-family:'Helvetica
Neue',Arial,sans-serif; line-height:1.7;
margin:0;">
```

```
        Do strength after an easy run or on
its own, never right before a long run or
quality session. If a session leaves you
sore for more than a day, drop the load
next time – the running always takes
priority over the lifting this close to
race day. Tell the coach if something hurts
rather than pushing through it.
```

```
      </p>
    </div>
```

```
  `;
}
```

```
function shiftStrengthWeek(n){
  window._strengthWeekOffset =
(window._strengthWeekOffset||0) + n;
  render();
}
```

```
const SHIFT_CYCLE =
```

```

['unknown','off','day','evening','night'];
function renderShifts(){
    window._shiftMonth = window._shiftMonth
    !== undefined ? window._shiftMonth : new
    Date().getMonth();
    window._shiftYear = window._shiftYear !==
    undefined ? window._shiftYear : new
    Date().getFullYear();
    if(!window._shiftStripAnchor)
    window._shiftStripAnchor = todayKey();
    const m = window._shiftMonth, y =
    window._shiftYear;
    const first = new Date(y, m, 1);
    const startOffset = (first.getDay()+6)%7;
    // Monday=0
    const daysInMonth = new Date(y, m+1,
    0).getDate();
    const monthName =
    first.toLocaleString('default',
    {month:'long', year:'numeric'});

    let cells = '';
    for(let i=0;i<startOffset;i++) cells +=
    `<div class="month-cell blank"></div>`;
    for(let d=1; d<=daysInMonth; d++){
        const date = new Date(y,m,d);
        const key = fmtDate(date);
        const isToday = key === todayKey();
        const done = isDayDone(key);
        cells += `<div class="month-cell
    ${isToday?'today':''} ${done?'done-
    day':''}" data-shift-cell="${key}">
        ${d}${dayDots(key)}
    </div>`;
    }
}

```

```

    }

    const strip = [];
    for(let d=1; d<=daysInMonth; d++){
        const date = new Date(y,m,d);
        const key = fmtDate(date);
        const isToday = key === todayKey();
        const done = isDayDone(key);
        strip.push(`<div class="day-chip
${isToday?'today':''} ${done?'done-
day':''}" id="stripChip-${key}"
onclick="openShiftPicker('${key}')">
        <div class="dow">${dowShort(date)}
</div>
        <div class="num">${d}</div>
        ${dayDots(key)}
</div>`);
    }
    const stripHtml = strip.join('');

    return `
        <h1 class="page-title">Shift
calendar</h1>
        <p class="page-sub">Tap a day to cycle:
not set → off → day → evening →
night</p>

        <div class="row" style="margin-
bottom:12px;">
            <button class="ghost"
onclick="shiftMonth(-1)">← Prev</button>
            <div style="text-align:center; align-
self:center; font-family:'Helvetica
Neue',Arial,sans-serif; font-

```

```

weight:600;">${monthName}</div>
    <button class="ghost"
onclick="shiftMonth(1)">Next →</button>
</div>

    <div class="shift-legend">

        ${Object.keys(SHIFT_LABELS).map(k=>`<span>
        <span class="dot"
        style="background:${SHIFT_COLORS[k]}">
        </span>${SHIFT_LABELS[k]}
        </span>`).join('')}</div>

        <div class="month-grid">

            ${['Mo','Tu','We','Th','Fr','Sa','Su'].map(
            d=>`<div class="dow-label">${d}
            </div>`).join('')}
                ${cells}
            </div>

            <div class="card">
                <h3>Quick edit – swipe and tap a
day</h3>
                <p style="font-size:12px; color:var(-
-lane-dim); font-family:'Helvetica
Neue',Arial,sans-serif; margin: 0 0 12px;">
                    Slide through the month and tap any
day to set it directly.
                </p>
                <div class="week-strip">${stripHtml}
            </div>
        </div>
    </div>

```

```
    ${window._shiftPickerDate ?  
renderShiftPickerSheet(window._shiftPickerDate)  
: ''}
```

```
    <div class="card">  
      <h3>Quick rotation (e.g. 4 on, 4 off)  
</h3>  
      <p style="font-size:13px; color:var(-  
-lane-dim); font-family:'Helvetica  
Neue',Arial,sans-serif; margin: 0 0 12px;">  
        Set your working pattern once and  
it builds the fill pattern below for you.  
      </p>  
      <div class="pill-row">  
        <div class="pill"  
onclick="setQuickRotation(4,4)">4 on / 4  
off</div>  
        <div class="pill"  
onclick="setQuickRotation(3,3)">3 on / 3  
off</div>  
        <div class="pill"  
onclick="setQuickRotation(2,2)">2 on / 2  
off</div>  
        <div class="pill"  
onclick="setQuickRotation(5,2)">5 on / 2  
off</div>  
      </div>  
      <div class="row">  
        <div><label>Days on</label><input  
id="qr_on" type="number" min="1" value="4">  
</div>  
        <div><label>Days off</label><input  
id="qr_off" type="number" min="1"
```



```

value="4"></div>
    <div><label>Shift while on</label>
    <select id="qr_type">
    <option
value="day">Day</option>
    <option
value="evening">Evening</option>
    <option
value="night">Night</option>
    </select>
    </div>
    </div>
    <button class="block"
onclick="buildQuickRotation()">Apply to
calendar</button>
    </div>

    <div class="card">
    <h3>Bulk fill a pattern</h3>
    <p style="font-size:13px; color:var(-
-lane-dim); font-family:'Helvetica
Neue',Arial,sans-serif; margin: 0 0 12px;">
    Set your rotation once – e.g. 4 on,
4 off – starting from a date, and it'll
fill forward.
    </p>
    <label>Start date</label>
    <input type="date" id="bulkStart"
value="${todayKey()}">
    <label>Pattern (comma separated,
repeats): off, day, day, day, day, off,
off, off, off</label>
    <textarea id="bulkPattern"
placeholder="off,day,day,day,day,off,off,of

```

```

f,off">off,day,day,day,day,off,off,off,off<
/textarea>
    <label>Weeks to fill forward</label>
    <input type="number" id="bulkWeeks"
value="12" min="1" max="52">
    <button class="block"
onclick="bulkFillShifts()">Fill
calendar</button>
</div>
`;
}
function setQuickRotation(on, off){
    document.getElementById('qr_on').value =
on;
    document.getElementById('qr_off').value =
off;
    buildQuickRotation();
}
function buildQuickRotation(){
    const on =
Number(document.getElementById('qr_on').val
ue) || 4;
    const off =
Number(document.getElementById('qr_off').va
lue) || 4;
    const type =
document.getElementById('qr_type').value;
    const pattern = [
        ...Array(on).fill(type),
        ...Array(off).fill('off')
    ].join(',');

    document.getElementById('bulkPattern').valu
e = pattern;

```

```

    // Apply immediately using whatever start
    date / weeks are set below, so one tap
    // actually updates the calendar rather
    than just previewing the pattern text.
    bulkFillShifts(`Rotation applied: ${on}
on (${type}) / ${off} off`);
}
function openShiftPicker(dateKey){
    window._shiftPickerDate = dateKey;
    window._shiftStripAnchor = dateKey;
    render();
}
function closeShiftPicker(){
    window._shiftPickerDate = null;
    render();
}
function pickShift(dateKey, type){
    if(type === 'unset') delete
state.shifts[dateKey];
    else state.shifts[dateKey] = type;
    saveState();
    window._shiftPickerDate = null;
    window._shiftStripAnchor = dateKey;
    render();
}
function renderShiftPickerSheet(dateKey){
    const d = parseKey(dateKey);
    const current = getShift(dateKey);
    const options = [
        { type:'off', label:'Off' },
        { type:'day',
label:`Day${state.settings.dayShiftHours?`
(${state.settings.dayShiftHours})`:''}` },
        { type:'evening',

```

```

label: `Evening${state.settings.eveningShift
Hours?}`
(${state.settings.eveningShiftHours})`:'`
},
    { type: 'night',
label: `Night${state.settings.nightShiftHour
s?}`
(${state.settings.nightShiftHours})`:'` }
];
return `
    <div class="modal-overlay"
onclick="if(event.target===this)
closeShiftPicker()">
        <div class="modal-sheet">
            <h3 style="margin:0 0
4px;">${dowShort(d)} ${monthDayLabel(d)}
</h3>
            <p style="font-size:12px;
color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-serif;
margin:0 0 16px;">Set this day's shift</p>
            ${options.map(o=>`
                <button class="block
${current===o.type ? ' ' : 'ghost'}"
style="margin-bottom:10px; text-
align:left;"
onclick="pickShift('${dateKey}','${o.type}'
)">
                    ${current===o.type ? '✓ ' :
''}${o.label}
                </button>`).join('')}
            <button class="ghost block"
onclick="closeShiftPicker()" style="margin-
top:6px;">Cancel</button>

```

```

        </div>
    </div>`;
}
function shiftMonth(n){
    let m = window._shiftMonth + n, y =
window._shiftYear;
    if(m<0){m=11;y--;} if(m>11){m=0;y++;}
    window._shiftMonth = m; window._shiftYear
= y;
    render();
}
document.addEventListener('click', (e)=>{
    const cell = e.target.closest('[data-
shift-cell]');
    if(cell){
        const key = cell.dataset.shiftCell;
        const cur = getShift(key);
        const idx = SHIFT_CYCLE.indexOf(cur);
        const next =
SHIFT_CYCLE[(idx+1)%SHIFT_CYCLE.length];
        if(next === 'unknown') delete
state.shifts[key];
        else state.shifts[key] = next;
        saveState();
        render();
    }
});
function bulkFillShifts(toastMsg){
    const startVal =
document.getElementById('bulkStart').value;
    const patternRaw =
document.getElementById('bulkPattern').valu
e;
    const weeksN =

```

```

Number(document.getElementById('bulkWeeks')
.value) || 12;
    const pattern =
patternRaw.split(',').map(s=>s.trim().toLow
erCase()).filter(s=>SHIFT_CYCLE.includes(s)
&& s!=='unknown');
    if(pattern.length === 0){ alert('Pattern
not recognised – use off, day, evening,
night separated by commas.');
```

return; }

```

    const start = parseKey(startVal);
    const totalDays = weeksN*7;
    for(let i=0;i<totalDays;i++){
        const d = addDays(start, i);
        const key = fmtDate(d);
        state.shifts[key] = pattern[i %
pattern.length];
    }
    saveState();
    showToast(typeof toastMsg === 'string' ?
toastMsg : `Filled ${weeksN} weeks of
shifts`);
    render();
}

/* ===== COACH VIEW ===== */
function renderCoach(){
    const hasKey = !!state.settings.apiKey;
    const chatHtml = state.chat.map(m=><div
class="msg
${m.role==='user'? 'user': 'coach'}">${escape
Html(m.content)}</div>`).join('');
    const voiceInputSupported = !!
(window.SpeechRecognition ||
window.webkitSpeechRecognition);

```

```

const voiceOutputSupported =
'speechSynthesis' in window;
return `
  <h1 class="page-title">Coach</h1>
  <p class="page-sub">Talk through how
training's going, adjust plans, or vent
about a rough shift</p>
  ${!hasKey ? `<div class="key-
warning">Add your Anthropic API key in
<button class="linklike"
onclick="setView('settings')">Settings</but
ton> to chat with your coach.</div>` : ''}
  <div class="pill-row">
    ${voiceInputSupported ? `<div
class="pill" style="cursor:default;">🎤 Tap
the mic to talk</div>` : ''}
    ${voiceOutputSupported ? `<div
class="pill
${state.settings.voiceReplies?'selected':''}
}"
onclick="toggleVoiceReplies()">${state.sett
ings.voiceReplies?'🎤':'🔊'} Voice
replies</div>` : ''}
  </div>
  <p style="font-size:12px; color:var(--
lane-dim); font-family:'Helvetica
Neue',Arial,sans-serif; margin: -8px 0
14px;">
    Tip: tap 📷 to attach a screenshot –
a Strava run, Garmin heart-rate/training
data, or a race result. The coach reads it
directly. Images aren't saved between
sessions, just used for that message.
  </p>

```

```

<div class="chat-log" id="chatLog">
  ${chatHtml || `<div class="empty">Say
hello – tell your coach how today's shift
went, how you're feeling, or attach a
screenshot of a recent run.</div>`}
</div>
  ${pendingImage ? `<div
style="display:flex; align-items:center;
gap:10px; margin-bottom:8px;">
    
    <span style="font-size:12px;
color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-
serif;">Screenshot ready to send</span>
    <button class="ghost"
style="padding:6px 10px; font-size:12px;"
onclick="clearPendingImage()">Remove</butto
n>
  </div>` : ''}
  <div class="chat-input-row">
    <button class="ghost" style="flex:0 0
auto; padding:11px 14px;"
onclick="attachScreenshot()"
${!hasKey?'disabled':''} title="Attach
screenshot"><img alt="camera icon" data-bbox="415 665 435 680"/></button>
    ${voiceInputSupported ? `<button
class="ghost ${window._recognizing?'mic-
active':''}" style="flex:0 0 auto;
padding:11px 14px;"
onclick="toggleVoiceInput()"
${!hasKey?'disabled':''} title="Voice
input">${window._recognizing ? '🗨️' : '🎤'}` : ''}

```



```

</button>` : ''}
    <textarea id="chatInput"
placeholder="${window._recognizing ?
'Listening...' : 'Message your coach...'}"
${!hasKey?'disabled':''}></textarea>
    <button onclick="sendChat()"
${!hasKey?'disabled':''}>Send</button>
</div>
    <input type="file" id="screenshotInput"
accept="image/*" style="display:none"
onchange="onScreenshotChosen(event)">
    `;
}
function toggleVoiceReplies(){
    state.settings.voiceReplies =
!state.settings.voiceReplies;
    saveState();
    if(!state.settings.voiceReplies &&
'speechSynthesis' in window)
window.speechSynthesis.cancel();
    render();
}
function speakText(text){
    if(!state.settings.voiceReplies || !
('speechSynthesis' in window)) return;
    window.speechSynthesis.cancel();
    const utter = new
SpeechSynthesisUtterance(text);
    utter.rate = 1.02;
    window.speechSynthesis.speak(utter);
}
function toggleVoiceInput(){
    const SR = window.SpeechRecognition ||
window.webkitSpeechRecognition;

```

```

    if(!SR){ showToast('Voice input not
supported in this browser'); return; }
    if(window._recognizing){
        window._recognitionInstance &&
window._recognitionInstance.stop();
        return;
    }
    const rec = new SR();
    rec.lang = navigator.language || 'en-GB';
    rec.interimResults = true;
    rec.continuous = false;
    rec.onstart = ()=>{ window._recognizing =
true; render(); };
    rec.onresult = (e)=>{
        let transcript = '';
        for(let i=0;i<e.results.length;i++)
transcript += e.results[i][0].transcript;
        const input =
document.getElementById('chatInput');
        if(input) input.value = transcript;
    };
    rec.onerror = ()=>{ showToast('Voice
input error – try again'); };
    rec.onend = ()=>{
        window._recognizing = false;
        const input =
document.getElementById('chatInput');
        if(input && input.value.trim()){
sendChat(); } else { render(); }
    };
    window._recognitionInstance = rec;
    rec.start();
}
let pendingImage = null;

```

```

function attachScreenshot(){
document.getElementById('screenshotInput').
click(); }
function onScreenshotChosen(e){
  const file = e.target.files[0];
  if(!file) return;
  const reader = new FileReader();
  reader.onload = ev => {
    pendingImage = { dataUrl:
ev.target.result, mediaType: file.type };
    render();
  };
  reader.readAsDataURL(file);
}
function clearPendingImage(){ pendingImage
= null; render(); }
function escapeHtml(s){
  const d = document.createElement('div');
  d.textContent = s;
  return d.innerHTML.replace(/\n/g,'<br>');
}

const COACH_TOOLS = [
  {
    name: "update_training_day",
    description: "Change what's planned for
a specific date in the athlete's training
app – use this whenever you and the athlete
agree on a change to their plan (they
missed a session, need a rest day, want to
swap a run, are injured, or you're
rescheduling around their shift). This
directly updates the app so the athlete
sees the new plan next time they open it.

```

```

Always call this rather than just
describing the change in words, whenever a
real plan change is agreed.",
  input_schema: {
    type: "object",
    properties: {
      date: { type: "string",
description: "The date to change, format
YYYY-MM-DD." },
      badge: { type: "string", enum:
["long", "quality", "easy", "cross", "rest", "st
rength"], description: "Category of the new
session." },
      description: { type: "string",
description: "Full plain-language
description of the new session, written
exactly as it should appear to the athlete,
e.g. 'Easy 4 miles, conversational pace –
making up for Tuesday's missed long run.'"
},
      miles: { type: "number",
description: "Distance in miles for this
session, or 0 if not a running session." }
    },
    required:
["date", "badge", "description"]
  }
};

function applyCoachToolCall(name, input){
  if(name === 'update_training_day'){
    const { date, badge, description, miles
} = input;

```

```

        if(!date || !/^d{4}-d{2}-
\d{2}$/.test(date)){
            return { ok:false, message: "Invalid
date format, must be YYYY-MM-DD." };
        }
        state.overrides[date] = { type: badge,
badge: badge || 'easy', desc: description,
miles: miles||0 };
        saveState();
        showToast(`Plan updated for ${date}`);
        return { ok:true, message: `Updated.
The athlete's app now shows on ${date}:
"${description}" (${badge}).` };
    }
    return { ok:false, message: `Unknown
tool: ${name}` };
}

function buildCoachSystemPrompt(){
    const s = state.settings;
    const race = getActiveRace();
    const key = todayKey();
    const eff = getEffectiveSession(key);
    const nutrition = nutritionForDay(key);
    const checkin = state.checkins[key] ||
{};
    const weeksOut = weeksToRace(race);
    const realism = raceRealism(race);
    const paces = trainingPaces(race);

    const recentCheckins =
Object.keys(state.checkins)
    .sort().slice(-4)
    .map(k=>`${k}: mood

```

```

    ${state.checkins[k].mood||'-'}/5, energy
    ${state.checkins[k].energy||'-'}/5,
    soreness ${state.checkins[k].soreness||'-'}/5${state.checkins[k].note?`, note:
    "${state.checkins[k].note}"`:'`')
    .join('\n');

    const otherRaces =
state.races.filter(r=>r.id!==race.id).map(r
=>`${r.name} (${r.date})${r.done?' -
completed':'`'}`).join(', ');
    const zones = hrZones();
    const zoneLines = zones ? ` - Heart rate
zones (Karvonen method, max ${s.maxHR}bpm /
resting ${s.restingHR}bpm): Z1
${zones.z1.lo}-${zones.z1.hi}, Z2
${zones.z2.lo}-${zones.z2.hi}, Z3
${zones.z3.lo}-${zones.z3.hi}, Z4
${zones.z4.lo}-${zones.z4.hi}, Z5
${zones.z5.lo}-${zones.z5.hi}bpm` : ` -
Heart rate zones not set up yet – encourage
them to add resting/max HR in Settings if
they start sharing Garmin heart-rate data';

    // Full current week, so the coach can
    see what's around today when rescheduling
    const ws = startOfWeek(new Date());
    const week = buildWeekSchedule(ws);
    const weekLines = week.map(d=>{
        const ov = state.overrides[d.dateKey];
        const sess = ov || d.session;
        const doneFlag = state.log[d.dateKey]
&& state.log[d.dateKey].done;
        const isPast = parseKey(d.dateKey) <

```

```

new Date(new Date().setHours(0,0,0,0));
    return `${d.dateKey}
    (${dowShort(d.date)}${d.dateKey===key?',
    TODAY':''}): ${SHIFT_LABELS[d.shift]} –
    ${sess.desc}${sess.strength ? ' + strength'
    : ''}${isPast ? (doneFlag ? ' [completed]'
    : ' [not marked done]') : ''}`;
  }).join('\n');

```

return `You are an elite marathon coach –
 the calibre of coach who has guided
 athletes to Boston-qualifying and sub-elite
 times – combined with a sports
 nutritionist, a strength coach for runners,
 and a supportive performance psychologist,
 working one-to-one with \${s.name || 'your
 athlete'}, age \${s.age}, training for the
 \${race.name} on \${race.date}.

COACHING METHODOLOGY – hold yourself to
 this standard

- Ground every recommendation in
 established endurance training science:
 progressive periodization (base → build →
 peak → taper), the roughly 80/20 easy-to-
 hard training distribution, and
 individualized pace prescription rather
 than generic advice.

- Long run and weekly mileage progression
 should never jump more than ~10% week over
 week; step-back recovery weeks every 3-4
 weeks are standard practice and already
 built into this athlete's plan.

- Taper science: volume typically drops 40-

60% over the final 2-3 weeks while intensity is largely maintained, so fitness is preserved without residual fatigue on race day.

- Use heart-rate zones (Karvonen method) when the athlete has HR data available, and pace-based prescription otherwise – both are valid, use whichever data they give you.

- Race-day fueling: general consensus is 30-60g carbohydrate per hour for efforts over 90 minutes, started early rather than waiting until you feel low; hydration should be guided by sweat rate and conditions, not a fixed number alone.

- Strength training for runners should prioritize single-leg stability, glute and hip strength, and general resilience over building bulk – this reduces injury risk without adding unwanted mass.

- Recovery (sleep, easy days actually being easy, deload weeks) is as much a part of the plan as the hard sessions – say so plainly when an athlete is at risk of under-recovering.

- Treat perceived exertion and how the athlete actually feels as real data, not something to override with the numbers on the plan.

ATHLETE PROFILE

- Weight: `${getWeightLb().toFixed(0)}lb`
(`${getWeightKg().toFixed(1)}kg`), height
`${s.heightCm}cm`

- Longest run to date before this plan:
`${s.longestRunMiles}` miles
- Preferred units: `${s.distanceUnit == 'km' ? 'kilometers / min per km' : 'miles / min per mile'}` – always answer in this unit unless they ask otherwise
`${zoneLines}`
- Works a rotating shift pattern (day/evening/night, varies) – training is built around whatever shifts they log in the app, which can change at short notice.
 Day shift: `${s.dayShiftHours || 'not set'}`.
 Evening shift: `${s.eveningShiftHours || 'not set'}`. Night shift:
`${s.nightShiftHours || 'not set'}`. On evening-shift days their morning is free for a proper run; day-shift days mean an early start so mornings are tighter; night-shift days should prioritise sleep over training.
- `${weeksOut}` weeks until race day
`${otherRaces ? '- Other races on file: ${otherRaces}` : ''}`

GOAL & REALISM

- Goal time: `${goalTotalMinutes(race) ? fmtMinutesAsHM(goalTotalMinutes(race)) : 'not set'}`
- Realism check: `${realism.text}`
`${paces ? '- Training paces: easy ${fmtPace(paces.easy)}, marathon ${fmtPace(paces.marathon)}, tempo ${fmtPace(paces.tempo)}, interval ${fmtPace(paces.interval)}` : ''}`

THIS WEEK'S PLAN (Mon-Sun, including any manual overrides already applied)
\${weekLines}

TODAY

- Shift: \${SHIFT_LABELS[getShift(key)]}
- Planned session: \${eff ? eff.session.desc : 'not generated'}\${eff && eff.session.strength ? ` + strength: \${eff.session.strength.exercises.join('; ')}' : ''}
- Check-in today: mood \${checkin.mood || 'not logged'}/5, energy \${checkin.energy || 'not logged'}/5, soreness \${checkin.soreness || 'not logged'}/5\${checkin.note ? `, note: "\${checkin.note}"` : ''}
- Fuel targets today:
~\${nutrition.totalKcal} kcal,
\${nutrition.carbsG}g carbs,
\${nutrition.proteinG}g protein,
\${nutrition.fatG}g fat,
\${(nutrition.waterMl/1000).toFixed(1)}L
water\${nutrition.needsElectrolytes ? ' plus electrolytes' : ''}

RECENT CHECK-INS

\${recentCheckins || 'None logged yet'}

HOW TO COACH

- Be direct, warm, and specific – like a real coach who knows this athlete, not a generic chatbot. Reference the actual

numbers above when relevant.

- When they're tired, sore, stressed by a shift, or low mood, take that seriously: adjust intensity recommendations accordingly and check in on how they're really doing before pushing training advice.

- Give concrete, actionable answers (paces, distances, strength exercises, specific food/hydration suggestions) rather than vague encouragement.

- If asked whether the goal time is realistic, use the realism check above as your starting point and explain the reasoning, don't just repeat the verdict.

- PLAN CHANGES: if the athlete tells you they missed, need to move, or want to change a session – decide with them whether to make it up, replace it, or just let it go, then use the `update_training_day` tool to actually apply the change to a specific date. Don't just talk about changing the plan – call the tool so the app reflects it. You can decide to keep the plan as-is and simply reassure them that's fine too; only call the tool when something should actually change.

- You are not a substitute for a doctor, physiotherapist, or a licensed therapist. If they describe symptoms of injury, disordered eating, or a mental health crisis, say so plainly and encourage them to see a professional, without being alarmist about normal training fatigue.

```
- Keep replies focused – a few short paragraphs or a tight list, not an essay, unless they ask for depth.`;  
}
```

```
async function sendChat(){  
  const input =  
document.getElementById('chatInput');  
  const text = input.value.trim();  
  if(!text && !pendingImage) return;  
  
  const displayText = (text || 'Sent a  
screenshot for you to look at.') +  
(pendingImage ? ' 📷 ' : '');  
  state.chat.push({role:'user', content:  
displayText, ts:Date.now()});  
  saveState();  
  
  let lastContent;  
  if(pendingImage){  
    const base64 =  
pendingImage.dataUrl.split(',')[1];  
    lastContent = [  
      { type:'image', source:{  
type:'base64', media_type:  
pendingImage.mediaType, data: base64 } },  
      { type:'text', text: text || "Here's  
a screenshot of my recent running/training  
data (could be Strava or Garmin) – read the  
relevant numbers (distance, time, pace,  
heart rate) and use them to judge my  
current fitness and recovery." }  
    ];  
  } else {
```

```

    lastContent = text;
  }
  const historyMessages =
state.chat.slice(0,-1).slice(-19).map(m=>
({role:m.role, content:m.content}));
  let workingMessages =
[...historyMessages, {role:'user',
content:lastContent}];

  input.value = '';
  pendingImage = null;
  render();
  const scrollLog = ()=>{ const el =
document.getElementById('chatLog'); if(el)
el.scrollTop = 999999; };
  scrollLog();

  const btn =
document.querySelector('.chat-input-row
button:last-child');
  if(btn){ btn.disabled = true;
btn.textContent = '...'; }

  let finalText = null;
  try{
    for(let guard=0; guard<4; guard++){
      const resp = await
fetch('https://api.anthropic.com/v1/message
s', {
        method: 'POST',
        headers: {
          'Content-Type':
'application/json',
          'x-api-key':

```

```

state.settings.apiKey,
      'anthropic-version': '2023-06-01',
      'anthropic-dangerous-direct-browser-access': 'true'
    },
    body: JSON.stringify({
      model: 'claude-sonnet-4-6',
      max_tokens: 800,
      system: buildCoachSystemPrompt(),
      messages: workingMessages,
      tools: COACH_TOOLS
    })
  });
const data = await resp.json();
if(data.error){
  finalText = `(Error from Claude API: ${data.error.message}||'unknown error'}). Check your API key in Settings.`;
  break;
}
const blocks = data.content || [];
const toolUses =
blocks.filter(b=>b.type==='tool_use');
const textBlock =
blocks.find(b=>b.type==='text');

if(toolUses.length === 0){
  finalText = textBlock ?
textBlock.text : '(No response text received.)';
  break;
}

```

```

        // execute each tool call locally,
        then continue the conversation with results

workingMessages.push({role:'assistant',
content: blocks});
    const toolResultContent =
toolUses.map(tu=>{
    const result =
applyCoachToolCall(tu.name, tu.input);
    return { type:'tool_result',
tool_use_id: tu.id, content: result.message
});
    });
    workingMessages.push({role:'user',
content: toolResultContent});
    render(); // reflect plan changes
immediately even before final text arrives
    }
    }catch(e){
        finalText = `(Couldn't reach the API:
${e.message})`;
    }
    if(finalText === null) finalText = 'Done
- updated your plan.';

    state.chat.push({role:'assistant',
content: finalText, ts:Date.now()});
    saveState();
    speakText(finalText);
    if(btn){ btn.disabled = false;
btn.textContent = 'Send'; }
    render();
    scrollLog();
}

```

```

/* ===== SETTINGS VIEW =====
*/
function renderSettings(){
  const s = state.settings;
  return `
    <h1 class="page-title">Settings</h1>
    <p class="page-sub">Your profile drives
every calculation in the app</p>

    <div class="card">
      <h3>Units</h3>
      <div class="pill-row">
        <div class="pill
${s.distanceUnit==='mi'? 'selected': ''}"
onclick="setDistanceUnit('mi')">Miles ·
min/mile</div>
        <div class="pill
${s.distanceUnit==='km'? 'selected': ''}"
onclick="setDistanceUnit('km')">Kilometers
· min/km</div>
      </div>
    </div>

    <div class="card">
      <h3>About you</h3>
      <label>Name</label>
      <input id="set_name"
value="${s.name}" placeholder="Matty">
      <div class="row">
        <div><label>Age</label><input
id="set_age" type="number"
value="${s.age}"></div>
        <div><label>Sex (for calorie calc)

```



```

</label>
    <select id="set_sex">
        <option value="male"
${s.sex==='male'? 'selected': ''}>Male</option>
        <option value="female"
${s.sex==='female'? 'selected': ''}>Female</option>
    </select>
</div>
</div>
<label>Height (cm)</label>
<input id="set_height" type="number"
value="${s.heightCm}">
<label>Weight</label>
<div class="row">
    <div><input id="set_stone"
type="number" value="${s.weightStone}"
placeholder="stone"></div>
    <div><input id="set_lb"
type="number" value="${s.weightLb}"
placeholder="lb"></div>
</div>
<label>Typical easy pace
(${s.distanceUnit==='km'? 'min per km': 'min
per mile'})</label>
<input id="set_pace" type="number"
step="0.1"
value="${paceMileToDisplayValue(s.easyPaceMinPerMile).toFixed(2)}">
</div>

<div class="card">
    <h3>Shift hours</h3>

```

```
<p style="font-size:13px; color:var(-lane-dim); font-family:'Helvetica Neue',Arial,sans-serif; margin: 0 0 12px;">
```

So the plan and coach know when you're actually free to train – e.g. an evening shift starting at 1pm still leaves the morning open for a run.

```
</p>
```

```
<label>Day shift hours</label>
```

```
<input id="set_dayhours"
value="${s.dayShiftHours}"
placeholder="7am-5pm">
```

```
<label>Evening shift hours</label>
```

```
<input id="set_eveninghours"
value="${s.eveningShiftHours}"
placeholder="1pm-11pm">
```

```
<label>Night shift hours</label>
```

```
<input id="set_nighthours"
value="${s.nightShiftHours}"
placeholder="11pm-7am">
```

```
</div>
```

```
<div class="card">
```

```
<h3>Heart rate zones (optional)</h3>
```

```
<p style="font-size:13px; color:var(-lane-dim); font-family:'Helvetica Neue',Arial,sans-serif; margin: 0 0 12px;">
```

Add these so the coach can read your Garmin heart-rate screenshots against real zones instead of guessing.

```
</p>
```

```
<div class="row">
```

```
<div><label>Max heart rate (bpm)
</label><input id="set_maxhr" type="number"
```

```

value="{{s.maxHR ?? ''}}"></div>
    <div><label>Resting heart rate
(bpm)</label><input id="set_restinghr"
type="number" value="{{s.restingHR ?? ''}}">
</div>
</div>
    {{hrZones() ? `<div style="font-
size:12px; color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-serif;
line-height:1.8;">
        Zone 1:
{{hrZones().z1.lo}}-{{hrZones().z1.hi}} ·
Zone 2:
{{hrZones().z2.lo}}-{{hrZones().z2.hi}} ·
Zone 3:
{{hrZones().z3.lo}}-{{hrZones().z3.hi}}<br>
        Zone 4:
{{hrZones().z4.lo}}-{{hrZones().z4.hi}} ·
Zone 5:
{{hrZones().z5.lo}}-{{hrZones().z5.hi}}
    </div>` : ''}
</div>

    <div class="card">
        <h3>Longest run & cross-training</h3>
        <label>Longest run completed so far
(miles)</label>
        <input id="set_longest" type="number"
value="{{s.longestRunMiles}}">
        <label>Preferred cross-
training</label>
        <input id="set_cross"
value="{{s.crossTrainDefault}}">
    </div>

```

```

    ${renderRacesCard()}

    <div class="card">
      <h3>AI Coach – API key</h3>
      <p style="font-size:13px; color:var(-
-lane-dim); font-family:'Helvetica
Neue',Arial,sans-serif; margin: 0 0 12px;">
        This app runs from a free static
        host, so coach chat calls the Anthropic API
        directly using your own key. It's stored
        only in this browser and never sent
        anywhere except Anthropic. Get a key at <a
href="https://console.anthropic.com"
target="_blank">console.anthropic.com</a>.
      </p>
      <label>Anthropic API key</label>
      <input id="set_apikey"
type="password" value="${s.apiKey}"
placeholder="sk-ant-...">
    </div>

    <button class="block"
onclick="saveSettings()">Save
settings</button>

    <div class="card" style="margin-
top:20px;">
      <h3>Data</h3>
      <p style="font-size:13px; color:var(-
-lane-dim); font-family:'Helvetica
Neue',Arial,sans-serif; margin: 0 0 12px;">
        Everything is stored in this
        browser only. Export a backup before

```

clearing browser data or switching devices.

```
    </p>
    <div class="row">
      <button class="ghost"
onclick="exportData()">Export
backup</button>
      <button class="ghost"
onclick="document.getElementById('importFil
e').click()">Import backup</button>
    </div>
    <input type="file" id="importFile"
accept="application/json"
style="display:none;">
  </div>
  `;
}
/* ===== RACES CARD (in Settings)
===== */
function renderRacesCard(){
  const active = getActiveRace();
  const rows = state.races.map(r=>`
    <div style="display:flex; justify-
content:space-between; align-items:center;
padding:8px 0; border-bottom:1px solid
rgba(232,226,212,0.08);">
      <div>
        <div style="font-
size:14px;">${r.name}${r.done?' ✓':''}
      </div>
      <div style="font-size:11px;
color:var(--lane-dim); font-
family:'Helvetica Neue',Arial,sans-
serif;">${r.date}</div>
    </div>
```

```

        <div style="display:flex; gap:6px;">
            ${r.id!==active.id ? `<button
class="ghost" style="padding:6px 10px;
font-size:12px;"
onclick="setActiveRace('${r.id}')">Set
active</button>` : `<span class="badge
quality">active</span>`}
            ${!r.done ? `<button class="ghost"
style="padding:6px 10px; font-size:12px;"
onclick="markRaceDone('${r.id}')">Done</but
ton>` : ''}
            ${state.races.length>1 ? `<button
class="ghost" style="padding:6px 10px;
font-size:12px;"
onclick="if(confirm('Delete this race?'))
deleteRace('${r.id}')">X</button>` : ''}
        </div>
    </div>`).join('');

    return `
    <div class="card">
        <h3>Races</h3>
        ${rows}
        <div style="margin-top:14px;">
            ${window._showAddRace ? `
                <label>Race name</label>
                <input id="new_race_name"
placeholder="Chicago Marathon">
                <label>Race date</label>
                <input id="new_race_date"
type="date">
                <div class="row">
                    <button
onclick="submitNewRace()">Add & make

```

```

active</button>
        <button class="ghost"
onclick="window._showAddRace=false;
render();">Cancel</button>
    </div>
    ` : `<button class="ghost block"
onclick="window._showAddRace=true;
render();">+ Add next race</button>`}
</div>
</div>

<div class="card accent">
    <h3>Active race: ${active.name}</h3>
    <label>Name</label>
    <input id="race_name"
value="${active.name}">
    <label>Date</label>
    <input id="race_date" type="date"
value="${active.date}">
    <label>Goal finish time</label>
    <div class="row">
        <div><input id="race_goalH"
type="number" placeholder="hours"
value="${active.goalHours ?? ''}"></div>
        <div><input id="race_goalM"
type="number" placeholder="minutes"
value="${active.goalMinutes ?? ''}"></div>
    </div>
    <label>Recent time-trial or race, for
a realism check (optional but recommended)
</label>
    <div class="row">
        <div><input id="race_recentDist"
type="number" step="0.1"

```

```

placeholder="distance (miles)"
value="${active.recentDistance ?? ''}">
</div>

    <div><input id="race_recentTime"
type="number" placeholder="time (minutes)"
value="${active.recentTimeMinutes ?? ''}">
</div>
    </div>
    <button class="block"
onclick="saveActiveRaceDetails()">Save race
details</button>
    </div>
`;
}
function submitNewRace(){
    const name =
document.getElementById('new_race_name').va
lue.trim();
    const date =
document.getElementById('new_race_date').va
lue;
    if(!name || !date){ alert('Add a name and
date. '); return; }
    addRace(name, date);
    window._showAddRace = false;
    showToast('New race added');
    render();
}
function saveActiveRaceDetails(){
    const r = getActiveRace();
    r.name =
document.getElementById('race_name').value.
trim() || r.name;
    r.date =

```



```
document.getElementById('race_date').value
|| r.date;
    const gh =
document.getElementById('race_goalH').value
;
    const gm =
document.getElementById('race_goalM').value
;
    r.goalHours = gh === '' ? null :
Number(gh);
    r.goalMinutes = gm === '' ? null :
Number(gm);
    const rd =
document.getElementById('race_recentDist').
value;
    const rt =
document.getElementById('race_recentTime').
value;
    r.recentDistance = rd === '' ? null :
Number(rd);
    r.recentTimeMinutes = rt === '' ? null :
Number(rt);
    saveState();
    showToast('Race details saved');
    render();
}

function setDistanceUnit(unit){
    state.settings.distanceUnit = unit;
    saveState();
    render();
}

function saveSettings(){
    const s = state.settings;
```

```

    s.name =
document.getElementById('set_name').value;
    s.age =
Number(document.getElementById('set_age').value)||s.age;
    s.sex =
document.getElementById('set_sex').value;
    s.heightCm =
Number(document.getElementById('set_height').value)||s.heightCm;
    s.weightUnit = 'stone';
    s.weightStone =
Number(document.getElementById('set_stone').value)||0;
    s.weightLb =
Number(document.getElementById('set_lb').value)||0;
    const paceInput =
Number(document.getElementById('set_pace').value);
    if(paceInput) s.easyPaceMinPerMile =
paceDisplayValueToMile(paceInput);
    s.maxHR =
document.getElementById('set_maxhr').value
=== '' ? null :
Number(document.getElementById('set_maxhr').value);
    s.restingHR =
document.getElementById('set_restinghr').value
=== '' ? null :
Number(document.getElementById('set_restinghr').value);
    s.dayShiftHours =
document.getElementById('set_dayhours').val

```

```

ue.trim();
    s.eveningShiftHours =
document.getElementById('set_eveninghours')
.value.trim();
    s.nightShiftHours =
document.getElementById('set_nighthours').v
alue.trim();
    s.longestRunMiles =
Number(document.getElementById('set_longest
').value)||s.longestRunMiles;
    s.crossTrainDefault =
document.getElementById('set_cross').value;
    s.apiKey =
document.getElementById('set_apikey').value
.trim();
    saveState();
    showToast('Settings saved');
    render();
}
function exportData(){
    const blob = new
Blob([JSON.stringify(state, null, 2)],
{type:'application/json'});
    const url = URL.createObjectURL(blob);
    const a = document.createElement('a');
    a.href = url; a.download = `split-coach-
backup-${todayKey()}.json`;
    a.click();
    URL.revokeObjectURL(url);
}
document.addEventListener('change', (e)=>{
    if(e.target.id === 'importFile'){
        const file = e.target.files[0];
        if(!file) return;

```

```
const reader = new FileReader();
reader.onload = (ev)=>{
  try{
    const parsed =
JSON.parse(ev.target.result);
    state =
Object.assign(defaultState(), parsed);
    saveState();
    showToast('Backup imported');
    render();
  }catch(err){ alert('Could not read
that file.');
```

```
};
  reader.readAsText(file);
}
});

function attachViewHandlers(){
  const chatInput =
document.getElementById('chatInput');
  if(chatInput){
    chatInput.addEventListener('keydown',
(e)=>{
      if(e.key === 'Enter' && !e.shiftKey){
e.preventDefault(); sendChat(); }
    });
  }
  const chatLog =
document.getElementById('chatLog');
  if(chatLog) chatLog.scrollTop =
chatLog.scrollHeight;

  const todayChip =
document.getElementById('todayChip');
```

```
    if(todayChip) todayChip.scrollIntoView({
      inline:'center', block:'nearest' });

    if(currentView === 'shifts' &&
      window._shiftStripAnchor){
      const anchorChip =
        document.getElementById('stripChip-' +
          window._shiftStripAnchor);
      if(anchorChip)
        anchorChip.scrollIntoView({
          inline:'center', block:'nearest' });
    }
  }

  /* ===== INIT ===== */
  // seed today's/upcoming shifts as unset by
  // default; nothing to do here.
  render();
```